

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 11.09.2026, 17:19:18
Файлов исходного кода: 87
Суммарный объём кода: 1.18 МВ
Глав/билетов в пособии: 30

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-dfs-bfs]
8. 7. Графы. Планарные. Покраска [id: graph-planarity-coloring]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sorting]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler-cycle]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-borushka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts
25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts
27. src/data/tickets/ticket14_20.ts
28. src/data/tickets/ticket21_24.ts
29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx
31. src/components/BellmanFordViz.tsx
32. src/components/BfsViz.tsx
33. src/components/ChapterImage.tsx
34. src/components/ChapterNav.tsx
35. src/components/ChapterView.tsx
36. src/components/DfsBridgesSimulator.tsx
37. src/components/DijkstraViz.tsx
38. src/components/DPVisualizer.tsx
39. src/components/EulerSimulator.tsx
40. src/components/ExpressQuiz.tsx
41. src/components/FloydViz.tsx
42. src/components/GraphTraversalViz.tsx
43. src/components/HeapViz.tsx
44. src/components/hints/demoKit.tsx
45. src/components/hints/demosExtra.tsx
46. src/components/hints/demosGraph.tsx
47. src/components/hints/demosStruct.tsx
48. src/components/hints/MiniDemo.tsx
49. src/components/hints/SimulatorModal.tsx
50. src/components/hints/TermPopover.tsx
51. src/components/JohnsonViz.tsx
52. src/components/KosarajuViz.tsx
53. src/components/KruskalSimulator.tsx
54. src/components/MnemonicCards.tsx
55. src/components/MobileToc.tsx
56. src/components/Navbar.tsx
57. src/components/PlanarityDemo.tsx

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

```
- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
```



```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
# На Ubuntu политика ImageMagick иногда запре
# разрешаем то, что нужно пайплайну (text -> p
for policy in /etc/ImageMagick-6/policy.xml /
    if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```

- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`\${public}/export/full\_code\_all\_pages`"
 echo "| \`\${public}/export/full\_code\_all\_pages`"

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DejaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```
-----  
    <body class="bg-slate-950 text-slate-100 min-h-screen  
....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg  
new file mode 100644  
index 00000000..3bbbba0  
--- /dev/null  
+++ b/public/favicon.svg  
@@ -0,0 +1,5 @@  
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
+</svg>  
....
```

```
## `src/App.tsx`
```

```
### Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from
import { chapters } from "../data/content";
```



```

 }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId}>
 </div>

 <main id="main-content" className="flex-1 min-w-0">
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index}>
 </div>
 </main>
 </div>
 </>
) : (
 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId}>
 </div>
 <main id="main-content" className="flex-1 min-w-0">
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index}>
 </div>
 </main>
 </div>
)}
 </Navbar>
 </div>
);

```

```

 </footer>
 </div>
);
}
....

Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
  </>
-  ) : (
+  ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
      </>
    </main>
  ) : (
    <main>
      <ChapterNav />
      <ChapterView />
    </main>
  )
}

```

```

<div className="flex items-center gap-3 w-full" >
  <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600 w-6 h-6" >
    <Sparkles className="w-6 h-6" />
  </div>
  <div className="min-w-0 flex-1" >
    <h1 className="text-xl font-extrabold text-slate-900" >
      Универсальное пособие
    </h1>
    <p className="text-xs text-indigo-400 font-medium" >
      Подготовка к экзаменам: Алгоритмы, Структуры данных
    </p>
  </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-slate-900 font-medium rounded-md shadow-sm transition-colors
      duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-slate-900 font-medium rounded-md shadow-sm transition-colors
      duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <CodeIcon className="w-4 h-4" /> Симулятор
  </button>
</div>

```

```

        </a>
        <button
          id="download-html-btn"
          onClick={saveBook}
          disabled={busy}
          className="flex items-center justify-center
            er:bg-amber-600/20 border border-amber-500/
          title="Скачать всё пособие одним автономным
        >
          {busy ? <Loader2 className="w-4 h-4 animate
        </button>
      </div>
    </div>
  </header>
);
};
....

```

Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

```

-----  
### Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}
```

```

        const captured = [...stdout, ...stderr].join("");
        setOutput(`${captured}${captured && !captured.end`
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >
                        Как это работает <ExternalLink className="w-4 h-4" />
                    </a>
                </div>
            </div>
        </div>
    </main>

```

```

    }}
    spellCheck={false}
    aria-label="Код Python"
    className="block w-full min-h-[360px] res
    -none focus:ring-2 focus:ring-inset focus:r
    placeholder="Напишите Python-код..."
  />

  <div className="border-t border-slate-800 b
    <label className="block px-4 pt-3 text-[1
      Ввод для input() • по одной строке на к
    </label>
    <textarea
      id="python-stdin"
      value={stdin}
      onChange={(event) => setStdin(event.tar
      spellCheck={false}
      rows={2}
      placeholder={'Например:\nАлиса'}
      className="block w-full resize-y bg-tra
    eholder:text-slate-700"
    />
  </div>
</section>

<section className="rounded-2xl border border
  <div className="flex items-center justify-b
    <div className="flex items-center gap-2 t
      <Terminal className="w-4 h-4 text-emera
    </div>
    <span className={`inline-flex items-cente
      {runtimeReady && <CheckCircle2 classNam
      {runtimeReady ? "готов" : "не загружен"}
    </span>
  </div>
  <pre className="min-h-[360px] max-h-[560px]
  x] leading-6 text-slate-300">
    {output}
  </pre>

```

Универсальное пособие • полный исходный код

```
-----  
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`  
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`  
+  
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код  
+name = "алгоритмы"  
+  
+for number in range(1, 4):  
+    print(f"{number}. Учим {name}!")  
+`;  
+  
+type PyodideRuntime = {  
+  runPythonAsync: (source: string) => Promise<unknown>  
+  setStdout: (options: { batched: (message: string) => void }) => void  
+  setStderr: (options: { batched: (message: string) => void }) => void  
+  setStdin: (options: { stdin: () => string | number | null }) => void  
+};  
+  
+type PyodideModule = {  
+  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>  
+};  
+  
+let runtimePromise: Promise<PyodideRuntime> | null = null;  
+  
+function getPythonRuntime() {  
+  if (!runtimePromise) {  
+    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {  
+      (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL })  
+    });  
+  }  
+  return runtimePromise;  
+}  
+  
+function errorText(error: unknown) {  
+  if (error instanceof Error) return error.message;  
+  return String(error);  
+}  
+  
+export function PythonCompiler() {
```



```

+
+  const reset = () => {
+    setCode(STARTER_CODE);
+    setStdin("");
+    setOutput("Нажмите «Запустить», чтобы увидеть резу.
+  };
+
+  return (
+    <main className="max-w-screen-2xl mx-auto px-4 sm:p
+      <div className="max-w-6xl mx-auto">
+        <div className="flex flex-col sm:flex-row sm:i
+          <div>
+            <div className="inline-flex items-center g
+              <Terminal className="w-4 h-4" /> Боковая
+            </div>
+            <h2 className="text-2xl sm:text-3xl font-e
+            <p className="text-slate-400 mt-2 max-w-2x
+              Пишите небольшой код и запускайте его пр
+              который переводит CPython в WebAssembly.
+            </p>
+          </div>
+          <a
+            href="https://pyodide.org/"
+            target="_blank"
+            rel="noreferrer"
+            className="inline-flex items-center gap-1.5
+          >
+            Как это работает <ExternalLink className="
+          </a>
+        </div>
+
+        <div className="grid xl:grid-cols-[minmax(0,1.
+          <section className="rounded-2xl border borde
+            <div className="flex flex-wrap items-cente
+              <div className="flex items-center gap-2"
+                <span className="w-2.5 h-2.5 rounded-f
+                <span className="text-sm font-bold tex
+                <span className="text-[11px] text-slate

```

```

+
+         <div className="border-t border-slate-800 p-4" style={{border: 1px solid #334155; border-top: none; margin-top: 10px;}}
+             <label className="block px-4 pt-3 text-slate-600" style={{display: block; padding: 0 10px 10px 10px;}}
+                 Ввод для input() • по одной строке на строку
+             </label>
+             <textarea
+                 id="python-stdin"
+                 value={stdin}
+                 onChange={(event) => setStdin(event.target.value)}
+                 spellCheck={false}
+                 rows={2}
+                 placeholder={'Например:\nАлиса'}
+                 className="block w-full resize-y bg-tran
+                 style={{border: 1px solid #334155; border-top: none; margin-top: 10px;}}
+             </div>
+         </div>
+     </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4" style={{border: 1px solid #334155; border-radius: 12px; margin-top: 10px;}}
+         <div className="flex items-center justify-between" style={{border: 1px solid #334155; border-radius: 12px; padding: 10px;}}
+             <div className="flex items-center gap-2" style={{border: 1px solid #334155; border-radius: 12px; padding: 10px;}}
+                 <Terminal className="w-4 h-4 text-emerald-500" style={{border: 1px solid #334155; border-radius: 12px; width: 100px; height: 100px;}}
+                 </div>
+                 <span className={`inline-flex items-center gap-2`} style={{border: 1px solid #334155; border-radius: 12px; padding: 10px;}}
+                     {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" style={{border: 1px solid #334155; border-radius: 12px; width: 100px; height: 100px;}}
+                     {runtimeReady ? "готов" : "не загружен"}
+                 </span>
+             </div>
+             <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" style={{border: 1px solid #334155; border-radius: 12px; padding: 10px;}}
+                 {output}
+             </pre>
+             <div className="border-t border-slate-800 p-4" style={{border: 1px solid #334155; border-top: none; margin-top: 10px;}}
+                 </div>
+         </section>
+     </div>
+
+     <div className="mt-5 grid md:grid-cols-3 gap-3" style={{border: 1px solid #334155; border-radius: 12px; padding: 10px;}}
+         <div className="flex gap-2 rounded-xl border

```

```

plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
  alias: {
    "@": path.resolve(__dirname, "src"),
  },
},
server: {
  host: "0.0.0.0",
  port: 5173,
  strictPort: true,
  // предпросмотр может открываться через внешний про
  allowedHosts: true,
},
preview: {
  host: "0.0.0.0",
  port: 4173,
  strictPort: true,
  allowedHosts: true,
},
});

```

Patch относительно `main`

```

````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {

```

Универсальное пособие • полный исходный код

- 
2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна иметь объект `{ "content": ... }` для вставки в ``src/data/content.ts``.
  3. **\*\*Строгая структура контента:\*\***
    - Заголовок с цветным бейджем (например, ``bg-indigo-100``)
    - Определение/правило в центре (``border-blue-500`` или ``border-blue-200``)
    - Обязательная сетка аналогий (2 колонки: неформальный текст и код)
    - "Душные" детали (код, формулы, сложные доказательства)
  4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы в файлах ``src/data/content.ts``. Не используй чистый `markdown`.

---

ФАЙЛ 4/87: `index.html`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

---

ФАЙЛ 5/87: `metadata.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

---

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "version": "1.0.0",
 "description": "A guide to the ExamPrep Reader, a tool for reading and understanding the ExamPrep Reader.",
 "keywords": "ExamPrep Reader, ExamPrep, ExamPrep Reader, ExamPrep Reader",
 "author": "Remix Remix: ExamPrep Reader",
 "license": "MIT",
 "repository": {
 "type": "git",
 "url": "https://github.com/remix-remix/exam-prep-reader"
 },
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview"
 },
 "dependencies": {
 "react": "^18.2.0",
 "react-dom": "^18.2.0",
 "react-router-dom": "^6.14.2",
 "vite": "^4.3.9"
 },
 "devDependencies": {
 "@types/react": "^18.2.14",
 "@types/react-dom": "^18.2.6",
 "@types/react-router-dom": "^6.14.2",
 "typescript": "^5.0.4"
 }
}
```

```
"devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

---

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

---

# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с поддержкой темной темы и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash
npm install
npm run dev          # предпросмотр на http://0.0.0.0:5174
npm run build        # сборка в dist/ (single-file)
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- \* **\*\*Содержание\*\*** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- \* **\*\*Переходы между темами\*\*** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src/`
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

## ## Структура

...

src/

- |                    |                                           |
|--------------------|-------------------------------------------|
| App.tsx            | – оболочка, привязка глав к визуализации  |
| components/        | – интерактивные симуляторы (Действия)     |
| data/              | – контент пособия (главы/билеты)          |
| scripts/export/    | – пайплайн код → txt → png → pdf          |
| public/export/     | – сгенерированные TXT и PDF (обновляемые) |
| .github/workflows/ | – автоматизация                           |

...

## # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.

Универсальное пособие • полный исходный код

```

> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет         | Тема                           | Статус                      |
|---------------|--------------------------------|-----------------------------|
| ---           | ---                            | ---                         |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. К                |
|               | ит вхолостую.                  |                             |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной гла                |
|               | огии <b>**</b> .               |                             |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть бе |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; к       |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- \* `stack-dfs` → `StackViz.tsx` – **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – **\*\*Очередь**
- \* `heap-beam-search` → `HeapViz.tsx` – **\*\*Куча и лучевой**
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – **\*\*Дер**
- \* `dynamic-programming` → `DPVisualizer.tsx` – **\*\*Динами**

|    |                                               |   |       |
|----|-----------------------------------------------|---|-------|
| 9  | 9. Графы. Топологическая сортировка           | 1 | -     |
| 10 | 10. Графы. Компоненты сильной связности (SCC) |   |       |
| 12 | 12. Графы. Точки сочленения                   | 1 | -   - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра          | 3 | -     |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман      | 1 |       |
| 16 | 16. Графы. Кратчайший путь. Флойд             | - | -     |
| 17 | 17. Графы. Алгоритм Джонсона                  | - | -     |
| 18 | 18. Графы. Остовное дерево. Краскал           | 1 | -     |
| 19 | 19. Графы. Остовное дерево. Прима             | 1 | -     |
| 20 | 20. Графы. Остовное дерево. Борувка           | 1 | -     |
| 21 | 21. Префикс-функция (π), КМП                  | 4 | -   - |
| 22 | 22. Z-функция                                 | 4 | -   - |
| 23 | 23. Алгоритм Ахо–Корасик                      | 2 | -   - |
| 24 | 24. Классы сложности, сведение задач          | 4 | -   - |
| -  | Обзор: Кратчайшие пути и Графы                | - | -   - |
| -  | Алгоритмы на карте                            | - | -   - |
| -  | Эйлер: Путь ≠ Цикл                            | - | -   - |
| -  | Мосты в графах: код + визуализация            | 1 | -     |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inli`
4. **\*\*Решить судьбу `alg-map`\*\*** — раскрыть или удалить.

ФАЙЛ 9/87: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
```



```

import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algov0/, локаль
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

```

const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {
 requestAnimationFrame(() => {
 document.getElementById("chapter-viz)?.scrollIntoView();
 });
 });
}, []);

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapter.length) * 100, [index, chapter.length]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar
 activeTab={activeTab}
 setActiveTab={setActiveTab}
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) => !o)}
 />

 <div className={compilerOpen ? "max-lg:pb-[58dvh]" : "max-lg:pb-[58dvh]"}>
 {activeTab === "guide" ? (
 <div>
 <MobileToc

```

```

 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 onOpenCompiler={openCompilerWithViz}
 />
 </main>
 </div>
</>
) : (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
)}

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное руководство
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

{compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 onOpenGuide={() => setActiveTab("guide")}
 onClose={() => setCompilerOpen(false)}
 />
)}
</div>

```

```
* , чтобы на
* повесить всплывающую карточку (как превью ссылок в В
*/
export function annotateTerms(root: HTMLElement): void {
 if (!cachedRegex) cachedRegex = buildRegex();
 const re = cachedRegex;

 const perTerm = new Map<string, number>();
 const perSurface = new Map<string, number>();
 const walker = document.createTreeWalker(root, NodeFilter.
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.
 if (!node.nodeValue || node.nodeValue.trim().length === 0)
 return NodeFilter.FILTER_ACCEPT;
 },
));

 const targets: Text[] = [];
 let current = walker.nextNode();
 while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
 }

 for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
```

```
 try {
 annotateTerms(el);
 } catch {
 /* если браузер не умеет lookbehind – просто оста
 }
 };

 // при смене главы
 // eslint-disable-next-line react-hooks/exhaustive-dep
 useEffect(run, deps);

 // страховка: контент перерисовали, а подсказок в нём
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
 });
}
```

---

ФАЙЛ 13/87: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 KB

---

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
```

```

 from {
 opacity: 0;
 transform: translateY(4px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
 }

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в В
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

```

```
.chapter-body :where(h2) {
 font-size: 1.25rem !important;
 line-height: 1.3 !important;
}
.chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
}
.chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
}
.chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}
```

```
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/87: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

---

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

---

ФАЙЛ 17/87: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМ

---



## Аналогия 2: Ступени

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded"
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приори
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border

```

```

 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
 }</pre>
</div>
</details>
</div>
</div>
</section>`,
},
{
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое чинит само себя",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">

 <!-- 0. Определение -->
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-lg sm:text-xl font-mono">
 поиска без
 : серия поворотов, которая поднимает v в кор
 <p class="text-sm text-slate-400 text-c">
 т – поворот.</p>
 </div>

```

```

 <ul class="list-disc list-inside text-slate-300">
 Важно!
 каждой вставки чинят дерево, чтобы оно не
 /li>
 Важно!
 менно быть кривым. Но каждый раз, когда мы
 ровнее становится.

 </div>

```

<!-- 2. Аналогии -->

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
 Аналогия 1: стопка бумаг на столе
 </div>
 <p class="text-slate-300 text-sm">У тебя
 скиваешь его и, поработав, кладёшь Важно!
 неделю самые нужные бумаги сами собой оказыва
 ет ровно это: «взял → положил наверх».</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
 Аналогия 2: тропинка через газон
 </div>
 <p class="text-slate-300 text-sm">Ты идёшь по дорожке, ко
 ода дорожка сама укорачивалась вдвое: чем чаще
 оплачивает все следующие. Пойдёшь один раз
 </p>
</div>

```

<!-- 3. Поворот -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border border-emerald-500 shadow-md">
 <h3 class="text-lg font-bold text-white mb-4">Поворот
 <p class="text-slate-300 text-sm mb-4">Поворот — это движение, три перевешенные ссылки и

```

```

 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
 <td class="py-3 pr-3 font-bolder">Зиг</td>
 <td class="py-3 pr-3">деда</td>
 <td class="py-3 pr-3">сам х</td>
 <td class="py-3">х стал королем</td>
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-3 pr-3 font-bolder">Пег</td>
 <td class="py-3 pr-3">х и п</td>
 <td class="py-3 pr-3">аво-право)</td>
 <td class="py-3">х поднялся</td>
 </tr>
 <tr>
 <td class="py-3 pr-3 font-bolder">Григ</td>
 <td class="py-3 pr-3">х и п</td>
 <td class="py-3 pr-3">Пег</td>
 <td class="py-3">п и г стали</td>
 </tr>
 </tbody>
</table>
</div>

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>
</div>

<!-- 5. Главная ловушка -->
<div class="bg-rose-950/30 p-6 rounded-xl border">
 <h3 class="text-lg font-bold text-rose-300">
 <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
 <div class="grid grid-cols-1 lg:grid-cols-2">

```

```
<p class="text-slate-300 text-sm mb-4">Дерево
3 шага, а потом делаем Splay(10).</p>
<pre class="bg-slate-950 p-4 rounded-lg overflow-x-
0 leading-relaxed">старт после Zi
 глубина 3 → 1 10 в корне
```

```
 40
 /
 30
 /
 20
 /
10
```

```
 30
 / \
 10 40
 \
 20
```

```
 10
 \
 30
 / \
 20 40
```

итог: было 4 узла в линию (высота 4) → стало высота 3,  
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути. Все узлы, мимо
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2
<h3 class="text-lg font-bold text-white mb-3">
<p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag
ах идея такая:</p>
<div class="grid grid-cols-1 md:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">c
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">H
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1">
 <p class="text-slate-400 text-xs">д
```

```

// один поворот: x встаёт на место своего родителя
function rotate(x) {
 const p = x.parent, g = p.parent;

 if (p.left === x) { // правый поворот
 p.left = x.right;
 if (x.right) x.right.parent = p;
 x.right = p;
 } else { // левый поворот (зеркало)
 p.right = x.left;
 if (x.left) x.left.parent = p;
 x.left = p;
 }

 p.parent = x; // родитель уехал вниз
 x.parent = g; // x занял его место
 if (g) { if (g.left === p) g.left = x; else g.right = x; }
}

// поднимаем x в корень
function splay(x) {
 while (x.parent) {
 const p = x.parent, g = p.parent;

 if (!g) { // ZIG: деда нет
 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сн
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сн
 rotate(x);
 }
 }
 return x; // теперь x —
}

// поиск: спустились и подняли найденное наверх

```

```

<details class="bg-slate-900/40 rounded-lg border-
 <summary class="p-4 font-bold text-slate-400
 open:border-b border-slate-700/50">
 Вопросы, которые задают на экзамене (
 </summary>
 <div class="p-5 text-sm text-slate-300 space-
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Последний
 енный или преемник). Splay делают всегда, да
 бы, и амортизация сломалась бы.</p>
 </div>
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Потому что
 еворачивают бамбук в другую сторону (см. бл
 </div>
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Чередовани
 вает один из них в корень, превращая дерево
 ступает только для одной конкретной
 </div>
 <div>
 <strong class="text-white block mb-
 <p class="text-slate-400">Treap дер
 play не хранит вообще ничего и держит балан
 p>
 </div>
 </div>
</details>

</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 }

```

```

 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic m
 <p class="text-xl font-mono text-white"
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

 description: "Разбиение орграфа на макро-вершины. А
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 нвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 БЫХ направлениях по односторонним улицам. Как это отразится
 у на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутые",
 content: `

```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-rose-300 italic"
 <p class="text-xl font-mono text-white"
тей > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4"
правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400

```

```

 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ней земли. Но если начать копать одновременно
 в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">

```

```

 </div>
 <p class="text-slate-300 text-sm mb-6">
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-6">
изкому племени для объединения. К вечеру пл

```

```

 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

description: "",
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 пример сортировка чисел). Класс NP —
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ь отличный переводчик на английский (Сведен
 аче B, то B — это NP-Полная (сосредо
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`

```

ин раз. Вершины повторять можно.</p>

```
<p><strong class="text-white">Эйлер
<div class="p-3 bg-slate-950 rounded-lg border-rose-500 shadow-md">
 <p class="text-sm text-blue-300">
 <p><strong class="text-white">Путь:
 <p><strong class="text-white">Цикл:
 </div>
</div>
</div>
```

```
<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла)

• шел-вошёл-вышел).

• финиш).
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-950 border-rose-500 shadow-md">
 Путь: Нормальная прогулка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная).
 Теплый пришёл в бар (вторая нечётная).
 Домой вернуться не смог, потому что
 </p>
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 border-rose-500 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь)
 </div>
 <p class="text-slate-300 text-sm mb-4">
```

```
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Мосты
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Мост: Единственная веревка

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Представь, что ты альпинист. Ты спустился в пещеру.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v]
 <p><strong class="text-white">low[v]
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold text-white">Мост: Единственная веревка
 <p class="text-xs text-slate-400">
 </div>
 </div>
 </div>
 </div>
```

```
<p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков
 Единственная ниточка. Порвётся — граф р
</p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
 <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спустился в пещеру.
 Из пещеры и нет других выходов — только веревка.
 Если верёвка (ребро v-u) оборвётся — граф р
```



```

int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}

```

</div>

<p class="text-xs text-slate-400">Занес

</div>

```
order-green-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `<code class="text-white">V=5, -white">10 &gt; 9</code>` – БАХ, непланарен!

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забу

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border
```

```
<div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `<code class="text-white">V=6`  
`xt-white">9 ≤ 12</code>` – проходит, но он в

Почему? Потому что у двудольного гр

Отсюда более строгое ограничение: `<`

`<code class="text-white">9 &gt; 8</`

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border
```

```
<summary class="p-4 font-bold text-slate-400
order-slate-700/50">
```

Доказательство непланарности К5 (Скры

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 spac
```

```
<p class="text-blue-400">Доказательство
```

```
<p>
```

Пусть К5 планарен.  $V = 5$ ,  $E = 10$ .

```
<p class="text-slate-300 text-sm">
 Важно: Это работает то
 епятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединены
 её перекроют (удалят вершину) – районы будут
 хрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
 <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
 абелем (мостом). Сами свитчи – это точки со
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
```

```
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 19/87: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);
```

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень) */
 func: string;
 /** Локальные переменные: имя → герг (усечённый). */
 locals: Record<string, string>;
}

export interface DebugResult {
 steps: DebugStep[];
 /** Текст последней необработанной ошибки (или пустая) */
 error: string;
 /** true, если уперлись в лимит шагов. */
 truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 500;

/**
 * Строит самодостаточный python-скрипт: выполняет userSrc
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
 */
export function buildDebugRunner(userSrc: string): string {
 const srcLiteral = JSON.stringify(userSrc); // JSON-сериализация
 return `import sys as __sys, json as __json`
```

```
`;
}
```

```
/**
```

```
 * Какие переменные изменились на этом шаге относительно
 * они подсвечиваются (как amber-подсветка изменений в
 * Новые переменные тоже считаются изменёнными.
 */
```

```
export function changedVars(
 prev: Record<string, string> | null | undefined,
 cur: Record<string, string>
): Record<string, boolean> {
 const out: Record<string, boolean> = {};
 for (const k of Object.keys(cur)) {
 out[k] = !prev || !(k in prev) || prev[k] !== cur[k]
 }
 return out;
}
```

```
/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name
```

```
/**
```

```
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
```

```
export function orderVars(
 cur: Record<string, string>,
 syncNames: string[]
): [string, string][] {
 const inSync = new Set(syncNames);
 const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
 const rest = Object.entries(cur)
 .filter(([k]) => !inSync.has(k))
 .sort(([a], [b]) => a.localeCompare(b));
 return [...syncEntries, ...rest];
}
```

```

 formal:
 "Очередь (FIFO). Кладём старт, достаём вершину –
 у рёбер.",
 complexity: "O(V + E)",
 demo: "bfs",
 simulator: "queue-bfs",
},
{
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину"],
 title: "DFS – обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся – откатываешься на
 formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
 complexity: "O(V + E)",
 demo: "dfs",
 simulator: "stack-dfs",
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого
 formal:
 "Полное бинарное дерево в массиве: дети узла i ле
 complexity: "вставка/извлечение O(log n), «посмотре
 demo: "heap",
 simulator: "heap-beam-search",
},
{
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь свер
 formal: "push / pop / top за O(1). На стеке живёт р
 demo: "stack",

```

```

 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то обход»)",
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксную ширину, то есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень), а потом

 кладём с в очередь.",
 complexity: "O(суммарной длины словаря × алфавит)",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки"],
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошла к

 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает суффикс",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топология"],
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая ссылка

 циклов.",

```



```

 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "деревья отрезков"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь массив, а его дети — за половинные куски.",
 formal: "Строится за $O(n)$, запрос и обновление — $O(\log n)$ ",
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается на две части.",
 formal: "Ребро (u, v) — мост, если $low[v] > tin[u]$, где $tin[u]$ — время захода в вершину u при обходе в глубину, а $low[v]$ — минимальное время захода в вершину v или её потомков (если ребро в родителя — нет).",
 complexity: " $O(V + E)$ одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "точки разрыва"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: увольь её — и коллапс.",
 formal: "Для не-корня: v — точка сочленения, если существует ребро (u, v) такое, что $low[v] > tin[u]$ ",
 demo: "articulation",
 simulator: "graph-articulation",
},

```

```

 demo: "np",
 },
 {
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»:
 formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$, где h – ра
 к путей сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
 },
 {
 id: "scc",
 labels: ["компонента сильной связности", "компоненты
 title: "Компоненты сильной связности",
 short:
 "Группы вершин, где из каждой можно дойти до кажд
 formal: "Косарайю: DFS с сохранением порядка выхода
 complexity: " $O(V + E)$ ",
 demo: "scc",
 simulator: "scc-kosaraju",
 },
 {
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу
 "префикс", "префикса", "суффиксом", "суффикс", "с
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начал
 от кусок.",
 formal:
 " $\pi[i]$ – длина наибольшего собственного префикса s
 возвращая указатель по тексту.",
 }

```

```

 formal:
 "Дейкстра — неотрицательные веса, $O(E \log V)$. Форд-Беллман, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
 },
 {
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикла"],
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»",
 formal:
 "Форд–Беллман: если на V -й итерации ещё удаётся с",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешёвое ребро", "самое дешёвое ребро", "самое дешёвое ребро",
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас",
 formal:
 "Корректность обычно доказывается «леммой о разрезе»",
 demo: "mst",
 simulator: "mst-kruskal",
 },
 {
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь",
 formal:

```

```
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс"],
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший к
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали кучу,
 complexity: "O(E log V) с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман"],
 title: "Алгоритм Форда–Беллмана",
 short:
 "Тупой, но честный: V–1 раз проходим по ВСЕМ рёбрам,
 ",
 formal:
 "После k-й итерации гарантированно верны все кратчайшие
 ось — есть отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд–Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки
 шить пересадку в k?",
 formal:
```

```

 "Сортировка $O(E \log E)$ + E операций union/find. C
complexity: "O(E log E)",
demo: "mst",
simulator: "mst-kruskal",
},
{
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "
 "компоненте связности", "связности", "связный", "
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мост
 бой.",
 formal:
 "Считаются одним проходом: пока есть непосещённая
complexity: "O(V + E)",
demo: "components",
simulator: "graph-dfs-bfs",
},
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "гра
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Горо
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улица
demo: "graph",
simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "с

```

```

 "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больш
 ый отрезали дважды.",
 formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
 complexity: "запрос O(1) после O(nm) предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведён
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно
 formal:
 "A ≤p B: есть полиномиальная функция, переводящая
 NP-полную задачу к новой.",
 demo: "np",
},
{
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива
 предподсчёты.",
 formal:
 "Суммы — префиксные суммы O(1). Минимум без изменен
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "с

```

```

 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в о
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
 demo: "zig-zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag — x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p — левый сын g, а x — прав
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева
 demo: "zig-zag",
 simulator: "splay-rotations",
 },
];

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) :

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string;
 e.labels.map((label) => ({ label, id: e.id })))
).sort((a, b) => b.label.length - a.label.length);

```

---

ФАЙЛ 22/87: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

---

```
import { Chapter } from "../types";
```

```
export const graphChapters: Chapter[] = [
```

```

 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Позитивное плани
 </div>
 <p class="text-slate-300 text-sm mb
 (нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstr
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb
 охождение улицы), Дейкстра сломается, так к
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <p>Алгоритм использует приоритетную
 g V). Гарантирует корректность только для н
 </div>
 </details>
```



```

 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-1">
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
 </div>
 </section>`
 },
 {
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Флойд
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg

```

```
<h3 class="text-xl font-bold text-blue-400 mb-4">

<div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">Осн
 <p class="text-lg font-mono text-white">Бор (Тр
</div>
<p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 у.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы двигаемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 терминальные ссылки (term_link) – прямые мос
 <p>
 </div>
 </div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 outl
 pen:border-slate-700/50">
```

```

 q.push(child);
 }
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <p class="text-slate-300 text-sm mb-4">Когда
 ут быть большими дробными числами (долгота
 ву от 0 до N, и уже по ним строим графы.</p>
 </div>
</div>
</section>`
}
];

```

```

 "Эйлера пути нет – нужно удалять лишние рёбра
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлерово
 ровно 2 нечётные или 0."
 }
],
"bridges-code": [
 {
 question: "После DFS у нас low[u] = 4, а tin[v] = 2",
 options: [
 "Да, так как low[u] (4) > tin[v] (2)",
 "Нет, low[u] должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: low[u] > tin[v]. В нашем
 },
 {
 question: "Для чего нужен массив low[v]?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого",
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: "low[v] хранит минимальное время входа в v
 поиска мостов и точек сочленения."
 },
 {
 question: "Какова временная сложность алгоритма поиска мостов",
 options: [
 "O(V^2) – ищем мосты в квадрате",
 "O(V + E) – каждый элемент один раз",
 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое ребро"
 }
]

```

ФАЙЛ 24/87: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 67 | РАЗМЕР: 1.5 КБ

```
import { createContext, useContext, useEffect } from "react";

/**
 * Связь «компилятор ↔ открытая визуализация».
 *
 * Панель компилятора в режиме пошагового отладчика транслирует
 * текущего шага трассы вместе с id текущей главы; пошаговые
 * главы (через useVizStepSync) подхватывают его и прыгают к нужной
 * визуализации идёт в ногу с листингом (каждая исполняемая строка
 * = очередной шаг подсветки, см. vizSync.ts).
 */

export const VizChapterContext = createContext("");

const CHANNEL = "algo:viz-step";

export interface VizStepEventDetail {
 chapterId: string;
 step: number;
}

/** Транслировать текущий шаг отладчика визуализациям трассы */
export function emitVizStep(chapterId: string, step: number) {
 if (typeof window === "undefined") return;
 window.dispatchEvent(new window.CustomEvent<VizStepEventDetail>(CHANNEL, {
 detail: { chapterId, step },
 }));
}

/**
 * Подписка визуализации на шаги отладчика.
 * Вызывается внутри компонента демонстрации:
 * useVizStepSync(currentStep, setCurrentStep, steps.length);
 */
export function useVizStepSync(currentStep: number, goNextStep: () => void, steps: number[]) {
 useEffect(() => {
 let currentStepIndex = 0;
 const nextStep = () => {
 currentStepIndex++;
 if (currentStepIndex < steps.length) {
 goNextStep();
 }
 };
 emitVizStep("", currentStepIndex);
 nextStep();
 }, [currentStepIndex, goNextStep, steps.length]);
}
```

```
/**
 * Захардкоженная «карта синхронизации» компилятора с визуализацией
 *
 * Для каждой страницы (главы) здесь вручную перечислены шаги
 * по которым идёт пошаговая подсветка в её интерактивной визуализации
 * в первую очередь индексы циклов i, j, k и размеры n, m, l
 *
 * Компилятор НЕ показывает код алгоритма – только комментарии
 * переменными, чтобы студент повторял шаги руками, глядя на код
 */
```

```
export interface SyncVariable {
 /** Имя переменной, как в коде визуализации: i, j, k, l, m, n */
 name: string;
 /** Роль в алгоритме (показывается во всплывающей подсказке) */
 role: string;
 /** Характерный диапазон/значение на демонстрации. */
 range?: string;
}
```

```
export interface PageSync {
 /** Заголовок демонстрации на странице (если интерактивна) */
 vizTitle?: string;
 /** Чему соответствует один шаг визуализации (для тултиза) */
 stepNote?: string;
 /** Переменные синхронизации. Пусто – на странице нет переменных */
 variables: SyncVariable[];
 /**
 * Минимальный исполняемый скелет для редактора: строка кода
 * шаг-в-шаг с подсветкой визуализации (без словесных комментариев)
 */
 code?: string;
 /**
 * Номера строк ВНУТРИ code (с 1), каждое выполнение строки
 * шаг визуализации: по ним отладчик точно переводит курсор
 * шаг демонстрации (иначе – грубо «строчный индекс = номер строки»)
 */
}
```

```

 variables: [
 { name: "x", role: "узел, который поднимаем к корню"},
 { name: "p", role: "родитель узла x перед поворотом"},
 { name: "g", role: "дедушка узла x – нужен в случае поворота"},
 { name: "zig / zag", role: "направление поворота: влево / вправо"},
],
 },
 "splay-rotations": {
 vizTitle: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
 stepNote: "Шаг = один кадр поворота из трёх классических",
 code: `x, p, g = "x", "p", "g" # узел, родитель, дедушка
 },
 print("zig: ", x, "↑", p)
 print("zig-zig: ", x, "↑", p, "↑", g)
 print("zig-zag: ", x, "↗", g)` ,
 variables: [
 { name: "x", role: "узел, который поднимаем к корню"},
 { name: "p", role: "родитель узла x перед поворотом"},
 { name: "g", role: "дедушка узла x – нужен в случае поворота"},
],
 },
 "sparse-table": {
 vizTitle: "Разрезанная таблица (1D и 2D)",
 stepNote: "Шаг построения = заполнение ячейки уровня",
 code: `n = 8 # размер массива
log = 3 # k ≤ log2(n)

1D: ячейка (i, j) – блок [i, i + 2^j)
i = 0
j = 0

2D: 4 ключа → одна ячейка
st2 = {} # (r, c, kx, ky) → ответ
r, c, kx, ky = 0, 0, 0, 0
st2[(r, c, kx, ky)] = 0

for j in range(1, log + 1):
 for i in range(n - 2 ** j + 1):

```

```

 { name: "memo[n]", role: "кэш уже посчитанных значений"},
],
},
"heap-beam-search": {
 vizTitle: "Куча (priority queue)",
 stepNote: "Шаг = просеивание: вставленный элемент в кучу",
 code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
 variables: [
 { name: "n", role: "текущее число элементов в куче"},
 { name: "i", role: "индекс элемента, который сейчас просеиваем"},
 { name: "(i-1)//2", role: "индекс родителя вершины"},
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка"},
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой вершины на стек или pop",
 code: `st = [] # стек = рекурсия
st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень выше
print("pop →", top, "стек:", st)`,
 variables: [
 { name: "top", role: "вершина стека — то, откуда мы вышли"},
 { name: "v", role: "текущая вершина графа, которую мы посещаем"},
],
},
"queue-bfs": {
 vizTitle: "Очередь и обход в ширину",
 stepNote: "Шаг = dequeue вершины из головы очереди",
 code: `from collections import deque

```



```

print(order)\,
 variables: [
 { name: "v", role: "текущая вершина обхода", range: 0..4},
 { name: "to / u", role: "очередной сосед вершины v", range: 0..4},
 { name: "visited", role: "множество уже посещённых вершин", range: 0..4},
 { name: "order", role: "порядок, в котором обход", range: 0..4},
],
},
"graph-components": {
 // отдельного виджета нет, тема опирается на DFS/BFS
 variables: [],
},
"top-sort": {
 vizTitle: "Топологическая сортировка",
 stepNote: "Шаг = выход рекурсии из вершины v: она добавлена в список",
 code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2], 4: [1]},
used, order = set(), []

def dfs(v):
 used.add(v)
 for to in adj[v]:
 # шаг: ребро v → to
 if to not in used:
 dfs(to)
 order.append(v)
 # выход из v!

for v in adj:
 if v not in used:
 dfs(v)
print(order[::-1])
разворот — ответ`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range: 0..4},
 { name: "to", role: "вершина, куда ведёт ребро из v", range: 0..4},
 { name: "i", role: "перебор стартовых вершин во вложенном цикле", range: 0..4},
 { name: "order", role: "список выхода из рекурсии", range: 0..4},
],
},
"scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)"

```

```

 { name: "i", role: "перебор стартовых вершин для "
 },
 },
 "bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v,
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

v, to = "B", "G" # шаг: возврат из G
is_bridge = low[to] > tin[v] # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед; ребро (v, to) проверя
 { name: "tin[v]", role: "время входа в вершину –
 { name: "low[v]", role: "минимум tin, куда можно
 { name: "timer", role: "глобальный счётчик времени
],
 },
 "euler-path-vs-cycle": {
 vizTitle: "Эйлеров путь и цикл",
 stepNote: "Шаг = один клик по следующей вершине мар
 code: `edges = [(0,1), (0,2), (1,3), (2,4), (1,2),
deg = {}
for u, v in edges: # степени вершин
 deg[u] = deg.get(u, 0) + 1
 deg[v] = deg.get(v, 0) + 1

odd = [v for v in deg if deg[v] % 2]
print(deg, "нечётных:", len(odd))
0 → цикл • 2 → путь • иначе – нельзя`,
 variables: [
 { name: "path", role: "текущий маршрут – последов
 { name: "last", role: "последняя вершина пути – о
 { name: "deg(v)", role: "степень вершины: чётная
],
 },

```

```

 heapq.heappush(pq, (dist[v], v))
print("итог:", dist)\,
 stepCodeLines: [4, 13, 17, 19, 22],
 variables: [
 { name: "u", role: "вершина с минимальным dist среди",
 { name: "v", role: "сосед вершины u, до которого проходим",
 { name: "w", role: "вес ребра (u, v)", range: "полюс",
 { name: "dist[v]", role: "лучшее известное расстояние от s до v"},
],
},
"bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
 code: `n, m = 7, 10 # вершин, рёбер
dist = {"S": 0}

for i in range(1, n): # итерация i = 1 ... n-1
 u, v, w = "S", "A", 5 # шаг: очередное ребро
 if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация
 if i == 1:
 print(f"итерация {i}: {dist}")
 break\,
 variables: [
 { name: "n", role: "число вершин графа – ровно n-1",
 { name: "m", role: "число рёбер, по которым проходим",
 { name: "i", role: "номер итерации (после i итераций)",
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес",
 { name: "dist[v]", role: "расстояние; если обновилось, то"},
],
},
floyd: {
 vizTitle: "Флойд–Уоршелл",
 stepNote: "Шаг = инициализация матрицы, смена промежуточного узла",
 code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
 [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0, INF],
 [INF, INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, INF, INF, 0],
 [INF, INF, INF, INF, INF, INF, 0]]

```

```

vizTitle: "Краскал и DSU",
stepNote: "Шаг = следующее ребро из отсортированных
code: `edges = [(3, "B", "C"), (1, "A", "B"), (2, "
edges.sort() # по весу

parent = {v: v for v in "ABC"} # DSU
def find(x):
 while parent[x] != x:
 x = parent[x]
 return x

mst = []
for w, u, v in edges: # шаг: очередное ребро
 if find(u) != find(v): # разные компоненты → берём
 parent[find(u)] = find(v)
 mst.append((w, u, v))
print(mst)\,
 variables: [
 { name: "n", role: "число вершин; в остов войдёт ",
 { name: "m", role: "число рёбер, сортируем по весу",
 { name: "i", role: "ребро сортируем по весу w; ра",
 { name: "parent[v]", role: "DSU: кто представител",
 { name: "rank[v]", role: "высота дерева DSU – по ",
],
},
"mst-prim": {
 vizTitle: "Прим",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

start = "A"
heap = [(0, start, "start")] # (вес, вершина, откуда)

w, v, parent = heapq.heappop(heap) # шаг: берём минимум
taken = {start} # дерево растёт от start
print(f"берём {v} за {w} (из {parent})")\,
 variables: [
 { name: "v", role: "вершина, которую только что д

```

```

 { name: "i", role: "индекс текущего символа – счи
 { name: "j", role: "длина текущего наилучшего сов
 { name: "π[i]", role: "префикс-функция: длина наи
 ица" },
],
},
"string-z-func": {
 vizTitle: "Z-функция",
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]
 code: `s = "abacaba"
n = len(s)
z = [0] * n
l = r = 0 # правый Z-блок [l, r]

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`,
 variables: [
 { name: "n", role: "длина строки s", range: "n = "
 { name: "i", role: "позиция, для которой считаем
 { name: "l, r", role: "правый крайний Z-блок: отр
 { name: "z[i]", role: "длина наибольшего общего п
],
},
"aho-corasick": {
 vizTitle: "Ахо–Корасик",
 stepNote: "Шаг = построение бора по образцам, затем
 code: `trie = {"": {}} # бор: вершина →
state, c = "", "a" # шаг: автомат идёт по симв

nxt = trie.get(state, {}).get(c, "") # go(state, c) с
print(f"δ({state!r}, {c!r}) → {nxt!r}")`,
 variables: [

```

```
}

=====
```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

```
=====
```

```

ФАЙЛ 26/87: src/data/tickets/ticket1_5.ts
```

```
КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМЕР: 1.5 КБ

```

```
import { Chapter } from '../..types';
```

```
export const tickets1to5: Chapter[] = [
```

```
{
```

```
 id: "segment-trees",
```

```
 title: "1. Дерево отрезков с операциями на отрезках",
```

```
 type: "html",
```

```
 description: "Дерево отрезков – структура данных для хранения информации о отрезках",
```

```
 category: "Оптимизации и Продвинутые структуры",
```

```
 content: `
```

```
<section id="segment-trees" class="mb-12 scroll-mt-10">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
 </div>
```

```
 <div class="space-y-8">
```

```
 <div id="intro" class="bg-slate-700/50 p-6 rounded">
```

```
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-blue-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 promise (lazy).</p>
```

```
 </div>
```

```

} </pre>
 </div>
 </details>
</div>
</div>
</section>
,
{
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 зок длины 7 (перекрывая друг друга по 1 единицу)
 ничего складывать, просто пересекаем куски.

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
 <h3 class="text-xl font-bold text-blue-400">Аналогия 1: Удар катаной (Split)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Аналогия 1: Удар катаной (Split)
 <p class="text-xl font-mono text-white">
 метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 по значению X. Все, кто меньше X улетают в
 </p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вого). Мы просто смотрим на корни: у кого Y
 как потомок.</p>
 </div>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
 <summary class="p-4 font-bold text-slate-300">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;

```



```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 вится корнем дерева (VIP-статус). Если ты член,
 ка почти не требуется времени!</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border-rose-500 shadow-md">
 Аналогия 2: Балансировка через
 </div>
 <p class="text-slate-300 text-sm mb-4">
 и становиться кривым как бамбук. Но как толк
 </p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
// Zig, Zag
function rotate(x) {
 let p = x.parent;
 if (p.left === x) { // Right rotate
 p.left = x.right;
 x.right = p;
 } else { // Left rotate
 p.right = x.left;
 x.left = p;
 }
}</pre>
 </div>
 </details>

 <div class="bg-indigo-950/60 p-6 rounded-xl">
 <h4 class="text-lg font-bold text-indigo">

```

```

<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 × Аналогия 1: Вырезание прямоугольника
 </div>
 <p class="text-slate-300 text-sm mb-4">
 з него маленький целевой прямоугольник в центре.
 ый верхний угол отрезается дважды! Поэтому
 </p>
 </div>
 </div>
 </div>
 </div>
 </div>
 </section>
}
];

```

ФАЙЛ 27/87: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМЕР: 1.2 KB

```
import { Chapter } from '../../../types';
```

```

 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
</section>`
},
{
 id: "bellman-ford",
 title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="bellman-ford-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15.
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-rose-400">
 <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана
 <p class="text-xl font-mono text-white">
 1. Инициализация
 2. Поиск кратчайшего пути
 3. Проверка отрицательного цикла
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Параноик проверяет ВСЕ возможные дороги V-1 раз подряд.
 </p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 ♂ Отрицательный цикл
 </div>
 </div>
 </div>
 </div>
 </div>
`

```

```

 <ul class="text-sm text-slate-300 space-y-4">
 Шаг k=0: Ты знаешь только начальный путь
 Шаг k=1: Разрешаем пересадки в начальный
е> через путь <code class="text-indigo-300">path[0]</code>
 Шаг k=2: Разрешаем пересадки в пути
нутри этих путей уже могут быть пересадки ч
 ...
 Шаг k=N: Ты проверил все возможные пути
 /li>

 </div>
 </div>
</section>`
},
{
 id: "johnson-algo",
 title: "17. Графы. Алгоритм Джонсона (Фокус с перевешиванием)",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Фокус
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы. Пути
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-amber-400">Алгоритм Джонсона
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">// Путь из s в t с минимальным перевешиванием
 </div>
 <div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-amber-500 shadow-md">
 ☞ Суть перевешивания
 </div>

```

```

 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"

```

```
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру племени
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
</div>
</section>`
 }
};
```

---

ФАЙЛ 28/87: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМЕР: 1.2 КБ

---

```
import { Chapter } from '../types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Префикс-функция (π) — Взгляд назад (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Строки
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">21. Префикс-функция (π) — Взгляд назад (КМП)</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-blue-400">Префикс-функция (π)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Префикс-функция (π) — это массив из целых чисел, где π[i] — это длина самого длинного
 <p class="text-xl font-mono text-white">сом, оканчивающимся в позиции i.</p>
 </div>
 </div>
 </div>
</section>`
 }
];
```

```

 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. <code>
 ab: Из начала ("a")
 >
 abc: Опять мимо. <code>
 abca: 0! Начало ("a"
 </code>
 abcab: Начало ("ab")
 /li>
 abcabc: Начало ("abc
 e>
 abcabcd: "d" всё исп
 "abca" vs "abcd"). Конец "abcd" не совпада

 </div>
 </details>
 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
 </div>
</div>
</section>`
 },
 {

```

## Аналогия 2: LLM Самокопиаст

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

В LLM Z-функция может применять  
[i]</code> (где <i>i</i> указывает назад на  
сама себя.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<p>Тот же пример: abcabcd</p>
```

```
<ul class="list-disc pl-5 space-y-2
```

```
<code>Z[0]</code> – не счита
```

```
<code>Z[1]</code> (bcabcd
```

```
[1] = 0</code>
```

```
<code>Z[2]</code> (cabcd
```

```

```

```
<code>Z[3]</code> (abcd
```

? Нет, в начале 'a', тут 'd'). Совпало 3 бу

```
<code>Z[4], Z[5], Z[6]</code>
```

```

```

```
</div>
```

```
</details>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
```

Код C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<pre class="bg-slate-950 p-4 rounded
```

```
int l = 0, r = 0;
```

```
for (int i = 1; i < n; i++) {
```



```

 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет — мы падаем по суффиксной ссылке ("
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 ou
 r-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-
 <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch)
 }
 return t[v].link;
}

int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v),
 }
 return t[v].go[c];
}

```

```
 <div class="absolute -top-3 left-4 border border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ь отличный переводчик на английский (Сведения о
 аче В, то В – это NP-Полная (сосредоточена на
 </div>
 </div>
</div>
</section>`
 }
};
```

---

ФАЙЛ 29/87: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">6. Графы. Представление, DFS, BFS
 </div>
 <div class="space-y-8">
 /* База представлений */
 </div>
</section>`
 }
];
```

```
<p class="text-slate-300 text-sm mb-4">
 Что делает: Жадный алгоритм находит ближайшую вершину (например, минимальную по номеру) и пробует другой путь.
```

```
</p>
<ul class="text-xs text-indigo-300 list-disc">
 Где используется:
 Топологическая сортировка (DFS)
 Поиск мостов и точек сочленения
 Сильно связанные компоненты Косараса
 Поиск циклов и просто проверка связности

</div>
```

```
<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
border border-emerald-500 shadow-md">
 BFS (Поиск в ширину) = Волна
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">
 Что делает: Концентрические волны от источника.
 Работает через Очередь (FIFO). Продвигается от источника.
```

```
</p>
<ul class="text-xs text-emerald-300 list-disc">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Алгоритм Ахо-Корасик (сборка слов)
 Алгоритм Форда-Фалкерсона / Поиск в ширину
 Двунаправленный поиск для ускорения

</div>
```

```
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
```

Код обходов C++ (Основа)

```

 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Теория
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Теорема о 4 красках
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Формулировка
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">где V — количество вершин, E — количество рёбер.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">Границы не пересекаются в одной точке по-
 тому что каждая граница имеет свой цвет, а цвета
 не сливались цветами, Всегда можно всего 4 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 content: `

```

```
<h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еская сортировка – это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 </div>
 <div class="bg-slate-900/40 rounded-lg p-6">
 <summary class="p-4 font-bold text-slate-300 border border-slate-700/50">
 Душный мод: Код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-indigo-400 font-bold">
 <div class="bg-slate-950 p-4 rounded">
```

```
<h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
```

```
<div class="space-y-8">
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
<h3 class="text-xl font-bold text-emerald-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
<p class="text-lg text-emerald-300 italic">
```

```
<div class="text-left font-mono text-sm">
```

```
<p><strong class="text-white">Компоненты ориентированного графа, в котором
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

```
«Это просто цикл?» – Нет!
```

```
</p>
```

```
<ul class="list-disc list-inside text-sm text-slate-300">
```

```
Если у тебя есть цикл <code>А → Б →
```

```
Если ты добавишь вершину Г
```

```
четыре вершины – одна большая SCC.
```

```
Правило: Если два цикла имеют хо
```

```

```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
```

```
<div class="bg-slate-800 p-6 rounded-lg">
```

```
<div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Интриги в офисе
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

```
Сильная связность – это "клуб" вершин в одной SCC. Добавить курьера Гошу, который может выйти наружу (к директору), но обратно в клуб не может.
```

```
</p>
```

```
</div>
```

```

 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-rose-400 mb-2">
 <div class="bg-slate-900 p-4 rounded-lg mb-2">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-2">
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутые",
 content: `

```

её перекроют (удалят вершину) – районы буд  
>хрупкость</strong> системы.

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border-  
<div class="absolute -top-3 left-4 bg-em  
er border-emerald-500 shadow-md">

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной  
абелем (мостом). Сами свитчи – это точки со

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border-  
<summary class="p-4 font-bold text-slate-400  
order-slate-700/50">

Душный мод: Код и корень DFS (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-

<p class="text-blue-400 font-bold">Особ

<p>

Для стартовой вершины обхода правил  
то выше неё прыгать некуда). Поэтому для не  
у него <strong>больше 1 независимого ребенка

</p>

<div class="bg-slate-950 p-4 rounded-lg

<pre class="text-xs font-mono text-

```
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;
```

```
 for (int to : adj[v]) {
 if (to == p) continue;
```





```

 { u: 3, v: 4 }, // 3 and 4 are articulation points {3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];
```

```
// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);
edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});
```

```
interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}
```

```
export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 useVizStepSync(currentStepIndex, setCurrentStepIndex,
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);
```

```

);

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",

```

```

 >
 {isPlaying ? {
 <Pause className="w-4 h-4 ml-1" />
 } : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 g
 <div className="lg:col-span-2 relative h-96 lg:l
 <svg className="w-full h-full" viewBox="0 0 8
 {/* Edges */}
 {edges.map((edge, i) => {
 const u = nodes.find((n) => n.id === edge
 const v = nodes.find((n) => n.id === edge

 const isBridge =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

 return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}

```

```

 className="transition-all duration-300"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
 >
 {node.label}
 </text>

 {isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0

```

```

 <div className="w-8 h-1 bg-rose-500 border"
 Мост
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2 rounded $
xt-slate-500`} `>
 >
 {step.desc}
 </div>
 </div>
 })}
 </div>
 </div>
 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded"
 <div
 className="bg-rose-500 h-full transition"
 style={{
 width: `${currentStepIndex / Math.ma
 }}
 />
 </div>
 </div>
</div>
</div>

```

```

 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start)' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations' },
 { line: 4, text: ' for (u, v, weight) in E:' },
 { line: 5, text: ' if dist[u] + weight < dist[v]' },
 { line: 6, text: ' dist[v] = dist[u] + weight' },

```

```

 }`,
 activeCodeLine: updated ? 6 : 5,
 });
}

if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: ` Ранний выход после итерации ${iter}.`,
 activeCodeLine: 10
 });
 break;
}
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount - 1) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}`,
 activeCodeLine: 9,
 });
 break;
 }
 }

 if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ` Проверка завершена. Ни одно ребро не попало в цикл.`,
 activeCodeLine: 10,
 });
 }
}

```





```

 ight="bold" textAnchor="middle" className="font-
 <text x={node.x} y={node.y + 35} fill="white">
 split(' ')[0]</text>
 </g>
 });
 }}}}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2 text-sm font-mono text-white">
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap gap-2">

 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === u;
 return (
 <span key={idx} className={`${px-2} ${
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
 } ${edge.v} ${edge.w}>

)
)
)
 }}}}
 </div>
)}}}
 </div>
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">

```

```
 });
 }
}
```

---

ФАЙЛ 32/87: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useCallback } from 'react';
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 },
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
 6: [];
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
}
```

```

 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
 });

 const neighbors = ADJ[curr];
 if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не`
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добав.`
 });
 }
 }
 } else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
 }
}

```

```

 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

return (
 <div className="bg-slate-900 border-2 border-slate-100" >
 <div className="flex flex-col md:flex-row items-center justify-center" >
 <div>
 <h3 className="text-2xl font-bold text-white" >
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Видео
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded" >
 <button
 onClick={() => { setIsPlaying(false); setStep(0); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-blue-300"
 title={isPlaying ? 'Пауза' : 'Воспроизведение'}
 >
 {isPlaying ? <Pause className="w-5 h-5" /> : <Play className="w-5 h-5" />}
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-emerald-300 disabled:opacity-50 disabled:cursor-not-allowed"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>
)

```

```

 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors durati
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
 }
 }
 </svg>
</div>

```

```

 { /* Код и Очередь */ }
 <div className="flex flex-col gap-4">

```

```

 <span className="text-slate-500 text-x
) : {
 step.queue.map(({qId, i}) => {
 <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

))
 })
</div>
<div className="mt-2 text-sm text-amber-300
ms-center justify-center">
 "{step.logs}"
</div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 33/87: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

```

```

const imageMap: Record<string, { src: string; alt: string;
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',

```

```

import { ChevronLeft, ChevronRight } from "lucide-react"
import type { Chapter } from "../types";

interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Далее
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 }
 return null;
};
```



```
 </button>
 })
 </nav>
);
};
```

---

ФАЙЛ 35/87: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, Loader2, Maximize2, MousePointer } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml";
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлены значения */
 onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && "ontouchstart" in window;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator, onOpenCompiler }) => {
 const contentRef = useRef<HTMLDivElement>(null);
```

```

 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect()
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap

```

```

ed-lg px-3 py-2">
<MousePointerClick className="w-4 h-4 shrink-0" />

 Подчёркнутые термины – интерактивные: наведи
 объяснение на пальцах, мини-анимацию и кноп

</p>

{viz && {
 <section id="chapter-viz" className="mt-8 scroll-mt-8" />
 <div className="flex items-center justify-between" />
 <h3 className="flex items-center gap-2 text-slate-600" />
 <Sparkles className="w-4 h-4 text-indigo-500" />
 Демонстрация: {viz.title}
 </h3>
 <div className="flex items-center gap-2" />
 <button
 onClick={() => onOpenSimulator(chapter)}
 className="flex items-center gap-1.5 text-slate-300
 hover:text-white hover:border-indigo-500" />
 <Maximize2 className="w-3.5 h-3.5" />
 </button>
 {onOpenCompiler && {
 <button
 onClick={onOpenCompiler}
 title="Открыть Python-компилятор: откройте комментарии"
 className="flex items-center gap-1.5 text-emerald-400
 hover:text-white hover:border-emerald-500" />
 <Terminal className="w-3.5 h-3.5" />
 </button>
 }}
 </div>
 </div>
 {viz.hint && <p className="text-xs text-slate-500" />
 <VizChapterContext.Provider value={chapter} />

```

```

{ line: "void dfs(int v, int p = -1) {", highlight: "normal" },
{ line: " visited[v] = true;", highlight: "normal" },
{ line: " tin[v] = low[v] = timer++;", highlight: "normal" },
{ line: " for (int to : adj[v]) {", highlight: "normal" },
{ line: " if (to == p) continue;", highlight: "normal" },
{ line: " if (visited[to]) {", highlight: "normal" },
{ line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
{ line: " } else {", highlight: "normal" },
{ line: " dfs(to, v);", highlight: "normal" },
{ line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " if (low[to] > tin[v]) {", highlight: "normal" },
{ line: " // ЭТО МОСТ!", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: "}", highlight: "normal" },
];

```

```

interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },

```

```

 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа A (уже посещён)! Обратное ребро",
 activeCodeLine: [7,8,9]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:2},
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:2,4:3},
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],

```

```

 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм завершён!",
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм завершён!",
 activeCodeLine: []
 },
],
];

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length - 1);
 const [dfsAuto, setDfsAuto] = useState(false);
 const dfsTimer = useRef<NodeJS.Timeout | null>(null);

 useEffect(() => {
 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
 }, [dfsAuto]);

 const step = DFS_STEPS[dfsIdx];

```

```

const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.active
 (step.activeEdge[0]===e.v && step.active
);
const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdge
 (step.backEdge[0]===e.v && step.backEdge
);
let sc = "#475569", sw = 2, dash = "none"
if (bridge) { sc = "#eab308"; sw = 4; }
else if (activeTree) { sc = "#10b981"; sw
else if (activeBack) { sc = "#f43f5e"; sw
else if (step.visitedIndices.includes(e.u
return <line key={e.key} x1={u.x} y1={u.y
 stroke={sc} strokeWidth={sw} strokeDash
 className="transition-all duration-500"
}})
{step.currentNode !== null && step.currentN
 const n = GRAPH_NODES.find(x => x.id ===
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} f
>;
}}())
{GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b
 stroke={cur ? "#fff" : st ? "#34d399"
 strokeWidth={2.5}
 className="transition-all duration-300
 <text x={n.x} y={n.y+3} textAnchor="mid
label}</text>
 {vis && (
 <
 <rect x={n.x-18} y={n.y-28} width={

```

```

 { /* Stack */ }
 <div className="bg-slate-900/50 rounded-xl bo
 <div className="text-[10px] font-bold text-
 Стек DFS
 {step.s
 </div>
 <div className="flex flex-col-reverse gap-1
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0
 i === step.stack.length-1
 ? "bg-emerald-600/20 text-emera
 : "bg-slate-900 text-slate-400
 }`}>Вершина {GRAPH_NODES.find(n=>n.
 <span className="text-slate-500 m
 </div>
 })
 }
 </div>
 </div>
 </div>
 </div>

```

```

 { /* Log */ }
 <div className="bg-indigo-950/10 border borde
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
 </div>
 </div>
</div>
</div>
</div>
);
}

```



```

 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

```

```

const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
});

```

```

const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbors(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push(v, dist[v])' },
 { line: 11, text: ' return dist // Все кратчайшие расстояния найдены' },
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
useVizStepSync(currentStepIndex, setCurrentStepIndex, setSteps);
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false, B: false };

```

```

 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: `    Улучшение (Релаксация)! Новое кратчайшее расстояние: ${dist[v]}.`,
 activeCodeLine: 9,
 });
 }
}

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: `    Дейкстра всё доставил! Все возможные кратчайшие расстояния найдены!`,
 activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

```

```

 className="flex items-center gap-1 bg-indigo-950/20
 px-3 py-2 rounded-lg text-sm font-medium tr
 >
 War
 </button>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml-6" >
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-indigo-950/20
 justify-center min-h-[400px]" >
 <div className="absolute top-3 left-3 bg-indigo-950/20
 font-bold shadow-sm" >
 War {currentStepIndex + 1} из {steps.length}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-dh-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#47556b" />
 </marker>
 <marker id="arrow-dh-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 { /* Рёбра */ }
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u)
 const vNode = nodes.find((n) => n.id === v)
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v)
 const isOptimal = currentStep.previous[ed

```

```

 return (
 <g key={node.id} className="transition-
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVis
 stroke={isCurrent ? '#a5b4fc' : isV
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-
 />
 <text x={node.x} y={node.y + 5} fill=
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill=
"middle" className="bg-indigo-950 px-1 py-0
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill=
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 }
 }
 </svg>
 <div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
 </div>
</div>

 /* Список смежности */
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.current
 return (
 <div key={u} className={`p-3 rounded-lg

```

```

<table className="w-full text-left border
 <thead>
 <tr className="border-b border-rose-900">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред.
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map(({n}) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentN === n.id

 return (
 <tr key={n.id} className={`border ${
 isCurr ? 'bg-indigo-950/60 font-medium' : ''
 }`} >
 <td className="py-2.5 px-3 flex items-center">
 <span className={`w-2 h-2 rounded-full ${
 vis ? 'bg-indigo-500' : 'bg-gray-300'
 }`} style={n.name}>
 </td>
 <td className="py-2.5 px-3 font-medium">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-gray-500">
 {prev || '-'}
 </td>
 </tr>
)
)
 })}
 </tbody>
</table>
</div>
</div>
</div>

```

```

 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' |
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}";
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 useVizStepSync(currentStepIndex, setCurrentStepIndex,
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibMemo(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }

```

```

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: memo[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
 }

 const finalRes = simulateFib(targetN);

 generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
}, [steps, speed]);

```

```

<div className="flex items-center bg-slate-950" style={{width: 1000px, height: 100px}}>
 <button
 onClick={() => { setIsPlaying(false); setSteps([0]); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); setSteps([0]); }}
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-2 py-2
 ${isPlaying ? 'bg-amber-500 text-slate-950 hover:bg-amber-600' : 'bg-emerald-600 text-white hover:bg-emerald-700'}
 `}
 >
 {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); setSteps([0]); }}
 disabled={currentStepIndex === steps.length - 1}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
</div>
</div>
</div>

```





```

 <div className="p-6 bg-slate-900/50 border-
 <span className="p-1.5 bg-emerald-500/20
 {currentStep?.desc}
 </div>
 </div>
)}
 </div>

 { /* Progress Bar */
 <div className="mt-8 pt-6 border-t border-slate-8
 <div className="flex items-center justify-between
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-transparent
 style={{ width: `${(currentStepIndex + 1) * 100}%` }}
 ></div>
 </div>
 </div>
 </div>
);
}

```

ФАЙЛ 39/87: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState } from "react";
import { Undo } from "lucide-react";

```

```

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },

```

```

 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 const startDegree = C1_EDGES.filter(e => e.u === vId).length;
 const endDegree = C1_EDGES.filter(e => e.v === vId).length;
 const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
 if (isCycle && nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
 } else {
 setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С`);
 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $`);
 }
 }
};

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>

 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)] border-[1px] border-dotted">
 <svg className="w-full h-[260px] z-10 relative">
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!;
 const v = C1_NODES.find(n => n.id === e.v)!;
 })}
 </svg>
 </div>
 </div>
 </div>
)

```

-----  
}

-----  
ФАЙЛ 40/87: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

-----  
import React, { useState } from 'react';  
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

```
interface Question {
 id: number;
 question: string;
 options: string[];
 correctAnswer: number;
 explanation: string;
}
```

```
const quizQuestions: Question[] = [
 {
 id: 1,
 question: 'Что мы делаем в алгоритме Краскала, если ребро соединяет две вершины, которые уже принадлежат одному дереву (т.е. оба конца уже красные)?',
 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево.',
 'Мы перекрашиваем их в синий цвет и делим граф пополам.',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл.',
 'Мы вызываем алгоритм Дейкстры для поиска нового минимального ребра.',
],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане, добавление ребра между ними создаст цикл. Это нарушит структуру дерева.',
 },
 {
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор на графе с отрицательными весами?',
 options: [
 'Потому что отрицательные веса нарушают предположение о том, что кратчайший путь найден.',
 'Потому что алгоритм не учитывает отрицательные веса.',
 'Потому что отрицательные веса создают бесконечные циклы.',
 'Потому что алгоритм не может найти минимальное ребро.',
],
 correctAnswer: 1,
 explanation: 'Алгоритм Дейкстры предполагает, что все веса неотрицательны. Если есть отрицательные веса, найденный путь может не быть кратчайшим, так как можно уменьшить его, пройдя через отрицательное ребро.',
 },
];
```

```
 options: [
 'СНМ (Система непересекающихся множеств / Disjoin',
 'Стек (LIFO).',
 'Красно-черное дерево.',
 'Хэш-таблица со списками коллизий.'
],
 correctAnswer: 0,
 explanation: 'Блестяще! СНМ (DSU) позволяет за прак
 .'
 }
];
```

```
export const ExpressQuiz: React.FC = () => {
 const [currentQIndex, setCurrentQIndex] = useState<num
 const [selectedOption, setSelectedOption] = useState<
 const [isAnswered, setIsAnswered] = useState<boolean>
 const [score, setScore] = useState<number>(0);
 const [showResults, setShowResults] = useState<boolean>
```

```
 const currentQ = quizQuestions[currentQIndex];
```

```
 const handleSelectOption = (index: number) => {
 if (isAnswered) return;
 setSelectedOption(index);
 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
 };
```

```
 const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
 }
}
```

```

 <RotateCcw className="w-5 h-5" />
 Пройти тест заново
 </button>
 </div>
);
}

return (
 <div className="bg-slate-900/90 border border-slate-
 >
 <div className="flex items-center justify-between
 <div className="flex items-center gap-3">

 Экспресс-тест

 <h3 className="text-xl font-bold text-white">
 </h3>
 </div>

 Bonpoc {currentQIndex + 1} / {quizQuestions.length}

 </div>

 <div className="mb-8">
 <h4 className="text-xl md:text-2xl font-bold text-white">
 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer;

 let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
 if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border border-emerald-500";
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-rose-500";
 }
 }
 return (
 <div className={optionStyle} key={idx} style="padding: 10px; border-radius: 5px; margin-bottom: 5px;">
 {option}
 </div>
);
 })}
 </div>
 </div>
 </div>
);
);
}

```

```

 <p className="text-slate-300 text-sm leading-
 {currentQ.explanation}
 </p>
 </div>
)}

 <div className="flex justify-end pt-4 border-t bo
 <button
 onClick={handleNext}
 disabled={!isAnswered}
 className={
 "px-8 py-3.5 font-bold rounded-xl transition
 (!isAnswered
 ? "bg-slate-800 text-slate-500 border bor
 : "bg-indigo-600 hover:bg-indigo-500 acti
 }
 >
 {currentQIndex + 1 < quizQuestions.length ? '
 </button>
 </div>
 </div>
);
};

```

ФАЙЛ 41/87: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import { useState, useEffect } from 'react';
import { useVizStepSync } from '../data/vizStepBus';

interface Step {
 k: number; i: number; j: number;
 matrix: number[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

```

```

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (999)',
 activeCodeLine: 2,
 });

 const N = 7;
 for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через пром`,
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({

```



```

<div className="p-2.5 bg-emerald-600 text-white" >

</div>
<div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Полн
 </p>
</div>
</div>

<div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false);
 text-sm"> Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying)}
 -amber-600' : 'bg-emerald-600'}`}>{isPlaying ?
 <button onClick={() => { setIsPlaying(false);
 ; }} className="px-3 py-2 bg-emerald-600 te
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml-6" >
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/20
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indigo-950/20
 ont-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ?
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-fw-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
 th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
 </defs>
 {Object.keys(adjList).flatMap((uStr) => {

```

```

 }}}
 </svg>

 <div className="w-full bg-slate-950/80 p-4 rounded-2xl" >
 <p className="font-semibold text-amber-400" >
 <p className="min-h-[40px] flex items-center" >
 </div>
 </div>

 { /* Список смежности */ }
 <div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl" >
 <h4 className="text-lg font-bold text-emerald-400" >
 <div className="space-y-2 text-sm font-mono text-white" >
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap` >
 <div key={u} className={`flex flex-wrap` >
 isI ? 'bg-amber-900/40 border-amber-400' :
 isK ? 'bg-indigo-900/40 border-indigo-400' :
 }` >
 →
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u;
 const isVia = currentStep.k !== u;
 const isDown = currentStep.j === edge.v);
 return (

 'down' : isVia ? 'bg-indigo-500 text-white border-indigo-400' :
 {edge.v} ({edge.w})

)
 })}
 </div>
)
 })}
 </div>
);
 }}}

```

```

 }}}
 </tbody>
 </table>
 </div>
</div>

 { /* Код алгоритма */
 <div className="w-full lg:w-1/2 bg-slate-900/60" >
 <div>
 <h4 className="text-lg font-bold text-slate" >
 <div className="bg-slate-950/80 rounded-lg p-4" >
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-4`}>
 e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'
 {item.line}
 {item.code}
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 42/87: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync } from '../data/vizStepBus';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';

```

```

type Node = { id: string; label: string; x: number; y: number; };
type Edge = { u: string; v: string };

```

```

 const stack: string[] = []; // for visualization, just for visualization

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Заши-
 itedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс
 .stack], visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Откат
 ueOrStack: [...stack], visitedNodes: Array.from(vis) });
 }

 dfs(start);
 return steps;
}

```

```

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начало
 ray.from(vis)] });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Действие

```



```

 className={`text-
 {n.label}
 </text>

 {/* Ripple for BFS */}
 {current && mode === 'bfs' ? (
 <circle cx={n.x} cy={n.y}
 className="stroke-emerald-500"
 r="10" />
) : null}
 </g>
)
 }}}}
 </svg>
 </div>

 {/* Structure View (Stack / Queue) */}
 <div className="bg-slate-900 border border-
 r">
 <div className={`absolute -top-3 left-3
 ${mode === 'dfs' ? 'border border-
 ld-300 border border-emerald-500'}`}>
 {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 </div>

 <div className="flex flex-col gap-2 p-3"
 {step.queueOrStack && step.queueOrStack.length > 0}
 >
 {step.queueOrStack.map((item, idx) => (
 <div key={idx} className={`w-full
 ${mode === 'dfs' ? 'bg-slate-800
 text-emerald-200' : 'bg-slate-700
 text-white'}
 padding-2px`}>
 {idx === (mode === 'dfs' ? step.queue.length - 1 :
 0_10px_rgba(245,158,11,0.2)]]' : ''}
 <div>
 {item}
 {idx === (mode === 'dfs' ? step.queue.length - 1 :
 0) ? (

 {mode === 'dfs' ? 'Вызван' : 'Выведен'}
) : null}
 </div>
)
)}
 </div>
 </div>
)}}}

```

```
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 43/87: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```

```
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}
```

```

 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩸' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' }
];

```

```

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болит сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩸' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' }
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();

```

```

let uid = 300;

```

```

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string[]>([]);
 const [shakeEmpty, setShake] = useState(false);
 const [busy, setBusy] = useState(false);
 const [healingPatient, setHealing] = useState<Patient>({});

```

```

 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

```

```

 // --- INSERT: Новый пациент поступает в приемный покой
 const insertPatient = useCallback((name: string, priority: number) => {
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выписки');
 setShake(true);
 }
 }, [busy, heap]);

```



```

};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(`СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 'winning' } : x));

 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 'exiting' } : x));

 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animStat: 'idle' }));
 });
 }, 100);
 }, 100);
}, [heap, busy]);

```

```

 }
 if (r < heap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
 });

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-emerald-500">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 text-center">
 Queue</h3>
 <p className="text-xs text-slate-300 mt-1 leading-relaxed">
 В реанимации не важно, кто пришел раньше, а кто
 остояние (максимальный приоритет).
 Куча (Heap) автоматически перестраивает пац
 оказывался на самом верху дерева (в корне).
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex flex-col lg:flex-row items-stretch">
 {/* Добавить случайного */}
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 >

```

```

 { /* Забрать самого тяжелого */ }
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r
 40 disabled:cursor-not-allowed text-white from-slate-800
 transition-all cursor-pointer flex items-center justify-center"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 border border-slate-700
 cursor-pointer"
 >
 Сброс
 </button>
 </div>

 { /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
 <div className="grid grid-cols-1 lg:grid-cols-12 gap-4">

 { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl min-h-[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`}>
 <div className="absolute top-4 left-4 text-[14px] font-medium">
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

 <div className="relative w-full h-full min-h-[300px]">
 <svg className="absolute inset-0 w-full h-full" style={{background: 'linear-gradient(to right, transparent 49%, #ccc 49% 51%, #ccc 51% 53%, transparent 53%)'}}>
 {edges}
 </svg>

 {heap.map((patient, idx) => {
 const pos = nodePos(idx);

```

```
</div>
```

```
{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
```

```
<div className="lg:col-span-4 bg-slate-900 rounded-3xl">
```

```
 <div className="absolute top-4 left-4 text-[10px] font-mono">
```

```
 Операционная койка
```

```
 </div>
```

```
<div className="flex-1 flex flex-col justify-between">
```

```
 {healingPatient ? (
```

```
 <div className="flex flex-col items-center">
```

```
 <div className="relative">
```

```
 {healingPatient.heart}
```

```
 {healingPatient.heart}
```

```
 {healingPatient.heart}
```

```
 </div>
```

```
 <div>
```

```
 <h4 className="text-white font-black">Пациент: {healingPatient.name}</h4>
```

```
 Пациент: {healingPatient.name}
```

```
 </div>
```

```
 <p className="text-emerald-400 text-x-small">Состояние: Стабилизация {healingPatient.status}</p>
```

```
 <p>Состояние: Стабилизация {healingPatient.status}</p>
```

```
 </div>
```

```
 <div className="flex justify-center gap-4">
```

```

```

```
 <Heart className="w-4 h-4 text-rose-500"></Heart>
```

```
 </div>
```

```
</div>
```

```
</div>
```

```
) : (
```

```
 <div className="text-center text-slate-600">
```

```
 {healingPatient.heart}
```

```
 <p className="text-xs font-bold font-mono">{healingPatient.heart}</p>
```

```
 <p className="text-[10px] mt-1">Ждём команду</p>
```

```
 </div>
```

```
 </div>
```

```
</div>
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```
/** Наведение на карточку ставит анимацию на паузу — можно
```

```
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {
```

```
 const [step, setStep] = useState(0);
```

```
 const paused = useContext(DemoPauseContext);
```

```
 const period = Math.round(ms * SPEED);
```

```
 useEffect(() => {
```

```
 if (steps <= 1 || paused) return;
```

```
 const t = setInterval(() => setStep((s) => (s + 1)),
```

```
 return () => clearInterval(t);
```

```
 }, [steps, period, paused]);
```

```
 return step;
```

```
}
```

```
export const C = {
```

```
 idle: "#334155",
```

```
 idleStroke: "#475569",
```

```
 text: "#e2e8f0",
```

```
 dim: "#94a3b8",
```

```
 active: "#6366f1",
```

```
 done: "#10b981",
```

```
 warn: "#f59e0b",
```

```
 bad: "#f43f5e",
```

```
 edge: "#475569",
```

```
};
```

```
export const Svg: React.FC<{ children: React.ReactNode;
```

```
 const paused = useContext(DemoPauseContext);
```

```
 color?: string;
 width?: number;
 dashed?: boolean;
 }> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);
```

---

ФАЙЛ 45/87: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7
 const pos: Record<string, [number, number]> = { i: [4, 4], j: [3, 3] };
 const viaOpen = step >= 1;
 const relaxed = step >= 2;
 return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь i → j = 9"
 : step === 1
```

```

const step = useLoop(comps.length + 1, 900);
const compOf = (v: string) => comps.findIndex((e) => e === v);

return (
 <Svg caption={step === 0 ? "Граф распался на острова" : "Граф соединился"}>
 {comps.flatMap((edges, ci) =>
 edges.map((([u, v], i) => (
 <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]} weight={w} />
)))
)}
 {Object.entries(pos).map((([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={k} />
 })}
 </Svg>
);
};

```

```

/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50], C: [250, 50], D: [350, 50], E: [450, 50], F: [550, 50], G: [650, 50], H: [750, 50], I: [850, 50], J: [950, 50] };
 const edges: [string, string, number][] = [
 ["A", "B", 1], ["B", "C", 1], ["C", "D", 1], ["D", "E", 1], ["E", "F", 1], ["F", "G", 1], ["G", "H", 1], ["H", "I", 1], ["I", "J", 1],
 ["A", "D", 2], ["B", "E", 2], ["C", "F", 2], ["D", "G", 2], ["E", "H", 2], ["F", "I", 2], ["G", "J", 2],
 ["A", "E", 3], ["B", "F", 3], ["C", "G", 3], ["D", "H", 3], ["E", "I", 3], ["F", "J", 3],
 ["A", "I", 4], ["B", "J", 4],
 ["A", "J", 5],
];
 return (
 <Svg
 caption={
 step === 0
 ? "Вершины – объекты (города, слова, состояния)"
 : step === 1
 ? "Рёбра – связи между ними"
 : "Вес ребра – цена перехода"
 }
 >
 {edges.map((([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 return (
 <g key={i}>

```

```

 : "Каждое число заменяем его номером → плотн
 }
>
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width
 <text x={startX + i * (boxW + 4) + boxW / 2}
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} hei
 <text x={53 + i * 52} y={77} textAnchor="mi
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
 raw.map((v, i) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} wi
 <text x={startX + i * (boxW + 4) + boxW / 2
 {sorted.indexOf(v)}
 </text>
 </g>
 })}
 </Svg>
);
};

/* ----- Splay: три случая поворота ---

type Pt = [number, number];

```



```

export const ZigDemo: React.FC = () => {
 <SplayFrames
 captions={["р – корень, х под ним", "Один поворот: "]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": [130, 32], "x": [78, 92]}],
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": [130, 32], "p": [182, 92]}],
]}
 />
};

/** Zig-Zig – х и р с одной стороны, крутим сначала верхнюю
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В"]}
 frames={[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112] }, edges: [{"g": [176, 24], "p": [126, 70]}, {"p": [126, 70], "x": [76, 112]}],
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78] }, edges: [{"p": [130, 26], "x": [78, 78]}, {"x": [78, 78], "g": [186, 78]}],
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112] }, edges: [{"x": [102, 24], "p": [152, 70]}, {"p": [152, 70], "g": [202, 112]}],
]}
 />
};

/** Zig-Zag – х и р с разных сторон, крутим сначала нижнюю
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions={["Зигзаг: g влево, x вправо", "Поворот НИ"]}
 frames={[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112] }, edges: [{"g": [176, 24], "p": [110, 70]}, {"p": [110, 70], "x": [152, 112]}],
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112] }, edges: [{"g": [176, 24], "x": [110, 70]}, {"x": [110, 70], "p": [64, 112]}],
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100] }, edges: [{"x": [130, 30], "p": [80, 100]}, {"p": [80, 100], "g": [180, 100]}],
]}
 />
};

```

```

 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) < 2;
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "black"} />
 })}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k === "носки" ? "red" : "black"} />
))}
 </Svg>
);
};

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["носки", "боты"],
 ["труссы", "боты"],
 ["штаны", "носки"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, [n.x, n.y]]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."} />
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed.indexOf(u) < placed.indexOf(v) ? "red" : "black"} />
))}
 {nodes.map((n) => {
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24} fill={idx <= 0 ? "red" : "black"} />
 <text x={n.x} y={n.y} textAnchor="middle" dx={0} dy={-10}>{n.id}</text>
 {idx >= 0 && <circle cx={n.x + 30} cy={n.y} r={10} fill="red" />
 {idx >= 0 && (
 <text x={n.x + 30} y={n.y - 12} textAnchor="left">{placed[idx]}</text>
)}
 </g>
);
 })}
);
};

```

```

<Svg caption={cut ? "Убрали C-D → граф развалился на 2" : "Убрали C → граф остался связным"}
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "black"} />
))}
</Svg>
);
};

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [["A", "B"], ["A", "C"], ["C", "D"]];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : "Убрали C-D → граф остался связным"}
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color="black" />))}
 {Object.entries(pos).map(([k, [x, y]]) =>
 cut && k === "C" ? (
 <circle key={k} cx={x} cy={y} r={12} fill="none" />
) : (
 <Node key={k} x={x} y={y} label={k} fill={k === "C" ? "red" : "black"} />
)
)}
 </Svg>
);
};

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["A", "D"], ["B", "C"], ["B", "D"], ["C", "D"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали ребро A-B → граф остался связным" : "Убрали ребро C-D → граф развалился на 2"}
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "D") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color="black" />))}
 {Object.entries(pos).map(([k, [x, y]]) =>
 cut && k === "C" ? (
 <circle key={k} cx={x} cy={y} r={12} fill="none" />
) : (
 <Node key={k} x={x} y={y} label={k} fill={k === "C" ? "red" : "black"} />
)
)}
 </Svg>
);
};

```

```

 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={step} />
 })}
 </Svg>
);
 };
};

```

ФАЙЛ 47/87: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit";

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [4, 8, 9, 12, 2]];
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68], [130, 112], [80, 156]];
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока не окажется на вершине" />
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b={pos[2]} />
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b={pos[2]} />
 {heap.map((v, i) => {
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={v} />
 })}
 </Svg>
);
};

```

```

export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], ["A", "B", "C", "D"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;

```

```
};
```

```
export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" >
 {p.map((par, i) =>
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
) : null
)}
 {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
 </Svg>
);
};
```

```
const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];
```

```
export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общий путь = префикс" >
 {TRIE_NODES.filter((n) => n.parent !== null).map((n) =>
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
)}
 </Svg>
);
};
```

```

 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x + 1) / 2, (a.y + 1) / 2} ${a.x + 1} ${a.y + 1}`}
 fill="none" stroke={C.warn} strokeWidth={1.5} />
);
 })}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f={n.f} />
))}
 </Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых" />
 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={20}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.warn} />
 <text x={n.x} y={n.y} textAnchor="middle" dom
 fill={C.idle} />
 </g>
))}
 </Svg>
);
};

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];

```

```

 <rect x={12 + i * 30} y={18} width={26} height={18} fill={C.done} />
 <text x={25 + i * 30} y={29} textAnchor="middle" font-size={14} />
 </g>
)}
 </Svg>
);
};

```

```

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $${avg}`}>
 {costs.map((c, i) => (
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18} fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} />
))}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
 <text x={130} y={20} textAnchor="middle" font-size={14}>Среднее: {avg}</text>
 </Svg>
);
};

```

```

export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svg caption={step === 0 ? "Найти решение – долго (N^2)" : "Решение найдено"}>
 <ellipse cx={130} cy={64} rx={112} ry={48} fill={step === 0 ? C.bad : C.done} />
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" />
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle" font-size={14}>N</text>
 <text x={206} y={34} textAnchor="middle" font-size={14}>P</text>
 <circle cx={196} cy={84} r={13} fill={step === 1 ? C.done : C.bad} />
 </Svg>
);
};

```

```

return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : fill}
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} text={C[idx]}
 fontSize={10} fontWeight="700" fill="black"/>
)}
 </g>
);
);
)}
)}
 <text x={130} y={122} textAnchor="middle" fontSize={14}
 каждая клетка считается один раз и переиспользуется </text>
 </Svg>
);
};

```

ФАЙЛ 48/87: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
 import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
 } from "../demosStruct";

```



```
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
 };

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
);
```

---

ФАЙЛ 49/87: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
```

```
interface Props {
 vizId: string | null;
 onClose: () => void;
}
```

```
 <div className="overflow-y-auto overscroll-cont
 <Component />
 </div>
 </div>
 </div>,
 document.body
);
};
```

---

ФАЙЛ 50/87: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}
```

```
interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}
```

```
const CARD_W = 320;
const GAP = 10;
```

```

if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500 p-3" >
 <div className="flex items-start gap-2 px-3 pt-3" >
 <h4 className="text-sm font-bold text-indigo-400" >Подсказка
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>
 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-slate-300" >
 {entry.demo && <MiniDemo kind={entry.demo} />
 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-slate-300" >
 {entry.formal}
 </p>
)}
 </p>
 </div>
 </div>
 >
 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-slate-300" >
 {entry.demo && <MiniDemo kind={entry.demo} />
 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-slate-300" >
 {entry.formal}
 </p>
)}
 </p>
 </div>
)

```

```
useVizStepSync(step, setStep, MAX_STEP);
```

```
const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];
```

```
const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];
```

```
const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
};
```

```
const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};
```

```
const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-width-2px";
 if (step === 2) return "stroke-pink-500 stroke-width-2px";
 }
 return "stroke-steelblue-500 stroke-width-2px";
};
```

```

const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0})`
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2})`
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1})`
 return e.newWeight;
 }
 return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
};

const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
};

const getDesc = () => {
 switch (step) {

```

```

 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$ <span cla
 </div>
);
 case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$ <span cla
 </div>
);
 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути оста
 </div>
);
 default: return "";
 }
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border">
 <h4 className="text-sm md:text-base font-bo">

 Визуализация: Перевзвешивание алгоритмом
 </h4>

 <div className="flex flex-col md:flex-row gap-4">
 <div className="bg-[#0f172a] flex-1 min-h-100 p-4">
 <svg width="400" height="300" class="dark">
 <defs>
 <marker id="arrow-slate" viewbox="0 0 100 100">

```

```

 <line
 x1={u.x} y1={u.y}
 className={`stroke-${u.id}`}
 markerEnd={getMarkerEnd(u.id)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="bottom"
 className={`text-${u.id}`}
 style={{
 textHeight: 1.2em,
 textEmergence: 1,
 isS ? 'fill:#2e3192' : 'fill:slategray'
 }}
 />
 {getEdgeContent(u, v)}
 </text>
 </g>
)
}}}

{/* Nodes */}
{nodes.map(n => {
 if (!isNodeVisible(n.id)) return null
 const pot = getPotential(n.id)

 return (
 <g key={n.id} className={`node-${n.id}`}>
 <circle
 cx={n.x} cy={n.y}
 className={`stroke-${n.id}`}
 style={{
 ${n.id === 'S' ? 'fill:#2e3192' : 'fill:slategray'}
 }}
 />
 <text x={n.x} y={n.y}
 style={{color: 'white'}}
 >{n.label}</text>
 </g>
)
})}

```

```

 <button onClick={() => setStep(
 className={`flex-1 font-bol
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:opacity-80'}
 {step === MAX_STEP ? "Готово" : "Сканирование"}
 {step !== MAX_STEP && <Skip>}
 </button>
 </div>

 {/* Progress Dots */}
 <div className="flex justify-center gap-2"
 {Array.from({length: MAX_STEP + 1}, (_, i) =>
 <div key={i} className={`w-12 h-12 rounded-full
 ${i < step ? 'bg-indigo-900' : 'bg-slate-200'}
 >
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 52/87: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import {
 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

```



```

 activeEdges: GEdge[];
 }

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

 const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
));
 };
 });
};

```

```
// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();
recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологическом порядке`,
 null,
 true,
 -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u)));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
```

```

 setSteps(generatedSteps);
 setCurrentStepIdx(0);
 }, []));

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

```

```

 { /* Markers for directed arrows */
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666"/>
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1f77b4"/>
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666"/>
 </marker>
 </defs>

 { /* Edges */

```

```

 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
 }
}

{/* Nodes */}
{initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node.id);
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "dfs2") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration-300 ${classes}`}
 strokeWidth="3"
 />
 </g>
);
})}

```

```

 { /* Logs & Stacks */ }
 <div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Порядок обхода и стек
 </h4>

 <div className="bg-slate-900 border border-slate-800 p-4">
 <p className="text-xs text-white leading-relaxed">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 { /* Top-Sort output / Order stack representation */ }
 <div>

 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-slate-900 p-4">
 {step.order.length === 0 ? (

 Нет элементов в стеке

) : (
 [...step.order].reverse().map(({val, idx}) => {
 <div
 key={idx}
 className="bg-indigo-950 border border-indigo-800
 px-2 py-1 text-xs text-white flex items-center gap-1"
 >
 {val}
 </div>
 })
)}
 </div>
 </div>

 { /* List of actions log */ }

```

```
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};
```

---

ФАЙЛ 53/87: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Checkmark } from 'lucide-react';

interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}
```

```
interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'rejected';
 color: string;
}
```

```
interface StepLog {
 step: number;
 action: string;
 details: string;
}
```

```
interface StepLog {
```

```

const [stepIndex, setStepIndex] = useState<number>(0)
const [heapQueue, setHeapQueue] = useState<string[]>()
const [activeCheatTab, setActiveCheatTab] = useState<

const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
]);

// --- KRUSKAL STEPS ---
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{

```



```

 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю – мы перекрашиваем всё в один цвет!"',
 type: 'warning'
 }, ...prev]);
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'blue' ? 'purple' : v.color));
 setEdges(prev => prev.map(e => e.status === 'included' ? 'excluded' : e.status));
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий кланы',
 type: 'success'
 }, ...prev]);
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B и E, которые уже соединены! Это создаст цикл!',
 type: 'warning'
 }, ...prev]);
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе появится цикл!',
 type: 'error'
 }, ...prev]);
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {status: 'excluded'} : e));
 setLogs(prev => [{

```

```

 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]));
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветн
 type: 'info'
 }, ...prev]));
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в осто́ве',
 description: 'Вершина H перекрашена в фиолетовы
 type: 'success'
 }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',

```

```

() => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-F (7)'],
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5']),
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
 description: 'Мы выбрали вершину E как стартовую точку и добавили в очередь (Heap): D-E(3), B-E(6), E-F(7)',
 type: 'info'
 }, ...prev]));
}, ...prev));
// Step 2: Pop D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
 description: 'Плесень тянется по ребру D-E к вершине E',
 type: 'info'
 }, ...prev]));
}, ...prev));
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ...e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-E (3)'],
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D: A-D(5), B-D(6), D-E(3)',
 type: 'success'
 }, ...prev]));
}, ...prev));
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вершины A'
 }, ...prev]));
}, ...prev));

```

```

 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет вершину E.',
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не может вернуться к B.',
 type: 'error'
 }, ...prev]));
},

```

```

 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]));
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I

```

```

 setHeapQueue(['F-I (13 - цикл)']);
 setLogs(prev => [{
 title: ' Успех: Плесень захватила весь граф
 description: 'Все 9 вершин поглощены. Итоговый
 иной – мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]);
 },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8
));
 setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбира
 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
 }, ...prev]);
 },
 // Step 2: Concurrently merge into Kolkhozes (Compo
 () => {
 // Concurrently build chosen roads. Merge into:

```

```

 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { .
 }));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский .
 description: 'Оба колхоза объединились по мосту
 type: 'success'
 }, ...prev]);
 },
 // Step 5: Check remaining edges & mark as rejected
 () => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шаг
 description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }, ...prev]);
 }
];

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);

```





```

 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
nitial cursor-pointer " +
 {stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
 }
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepI
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
 /* Graph Canvas */

```



```

 <p className={"font-bold text-base " +
ld-300' : 'text-rose-300'}}>
 MST построено с помощью {activeAlgo}
 </p>
 <p className="text-slate-300 text-xs mt
 Общий вес минимального остова: 51. Вс
 </p>
 </div>
 })
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl border
 <div className="p-4 bg-slate-900 border-b b
 <h4 className="font-bold text-slate-200 t
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && {
 <div className="p-3 bg-slate-900/60 borde
 <p className="text-[11px] font-bold tex
 ⚡ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5"
 {heapQueue.length === 0 ? {
 <span className="text-xs text-slate
) : {
 heapQueue.map(({item, idx}) => {
 <span key={idx} className={"px-2 p
0 shadow-md animate-pulse" : "bg-slate-800
 {item}

 })
 })
 </div>
 </div>
 })
}
```

```

<div className="flex bg-slate-950 p-1 rounded
 <button
 onClick={() => setActiveCheatTab('kruskal')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-950 text-slate-400"
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'prim' ? "bg-emerald-600 text-white" : "bg-slate-950 text-slate-400"
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-950 text-slate-400"
 >
 Борувка (Билет 20)
 </button>
</div>
</div>

```

```

{activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols-2"
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-indigo-500" />
 </p>
 <p className="text-2xl font-black text-white">Краскал</p>
 <p className="text-slate-400 text-xs font-medium">Минимальное остовное дерево</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">

```

```

 </pre>
 </div>
 </div>
 </div>
})

```

```

{activeCheatTab === 'prim' && {
 <div className="grid grid-cols-1 lg:grid-cols-2">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-emerald-400"/>
 </p>
 <p className="text-2xl font-black text-white">Примитивы</p>
 <p className="text-slate-400 text-xs mt-2">Примитивы — это базовые типы данных, которые используются для хранения информации. Они являются фундаментом любой программы.
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Award className="w-3.5 h-3.5 text-emerald-400"/>
 </p>
 <p className="text-2xl font-black text-white">Награды</p>
 <p className="text-slate-400 text-xs mt-2">Награды — это способ поощрить пользователя за достижение определенных целей. Они могут быть в виде значков, медалей или просто текста.
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-emerald-400"/>
 </p>
 <p className="text-2xl font-black text-white">Код</p>
 <p className="text-slate-400 text-xs mt-2">Код — это набор инструкций, которые компьютер может выполнять. Он является основой любой программы.
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-emerald-400"/>
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">

```

```
{`import heapq`}
```

```

 <p className="text-slate-400 text-xs mt-4">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 ении:</p>
 <p className="text-xs text-slate-300 leading-5">
 , организовав массовое распараллеленное слияние
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-rose-500">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">
def boruvka(n, edges):
 parent = list(range(n))
 mst = []
 while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2] > w:
 cheapest[ru] = (u, v, w)
 if cheapest[rv] == -1 or cheapest[rv][2] > w:
 cheapest[rv] = (u, v, w)
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge[1]):
 mst.append(bridge)
 </pre>
 </div>
 </div>
</div>
 })
</div>

```

```

 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Все-ко-Все Флойд',
 title: ' Все ко Все (Флойд)',
 desc: 'Матрица + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
 rest: '). Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-500/30',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover grid" />
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.highlightColor}`>{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc} {c.highlight}
 </p>
 <div className={`mt-3 text-xs font-mono px-1`>{c.complexity}</div>
 </div>
 </div>
))}
 </div>
);
};

```

```
return () => window.removeEventListener("keydown",
}, []);
```

```
return (
```

```
 <
```

```
 <div className="lg:hidden sticky top-[57px] z-40"
```

```
 <button
```

```
 onClick={() => setOpen(true)}
```

```
 className="w-full flex items-center gap-2.5 p-2"
```

```
 aria-label="Открыть содержание"
```

```
 >
```

```
 <span className="p-1.5 rounded-lg bg-indigo-600"
```

```
 <List className="w-4 h-4" />
```

```

```

```

```

```
 <span className="block text-[10px] uppercase"
```

```
 Содержание • {index + 1} из {total}
```

```

```

```
 <span className="block text-[13px] font-bol"
```

```

```

```
 </button>
```

```
 </div>
```

```
{open && (
```

```
 <div className="lg:hidden fixed inset-0 z-[90]"
```

```
 <div className="absolute inset-0 bg-slate-950"
```

```
 <div className="absolute inset-y-0 left-0 w-[100%]"
```

```
 te-[tocIn_.18s_ease-out]">
```

```
 <div className="flex items-center justify-b"
```

```
 <span className="font-bold text-white tex"
```

```
 <button
```

```
 onClick={() => setOpen(false)}
```

```
 className="p-2 -mr-2 text-slate-400 hov"
```

```
 aria-label="Закрыть содержание"
```

```
 >
```

```
 <X className="w-5 h-5" />
```

```
 </button>
```

```
 </div>
```



```

 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };

 return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >

```

```

 >
 <FileDown className="w-4 h-4" /> Скачать PD

 <a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 :bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
</header>
);
};

```

---

ФАЙЛ 57/87: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

export function PlanarityDemo() {

```

```

 if (intersect(edges[i][0],edges[i][1])
 if (!crosses.includes(i)) crosses.push(i)
 if (!crosses.includes(j)) crosses.push(j)
 }
 }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x}
 stroke={xing ? "#f43f5e" : "#4f46e5"} />);
}
const labels = ["A","B","C","D","E"];
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
 <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
 </g>);
}
return <>{lines}{circles}</>;
})()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border-1">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
]
 const edges = [
 [0,1],[1,2],[2,0],[3,4],[4,5],[5,3],
 [0,3],[0,4],[1,5],[1,3],[2,5],[2,4],
]
 const crosses = [0,1,2,3,4,5]
 return <>{lines}{circles}</>;
 })()}
 </svg>

```

```

 return <>{lines}{circles}</>;
 }}())}
 </svg>
 <div className="absolute bottom-2 left-2 right-2
 z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 58/87: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useCallback, useEffect, useМemo, useRef, useSt
import {
 ArrowRightLeft,
 Bug,
 Check,
 CheckCircle2,
 ChevronLeft,
 ChevronRight,
 Copy,
 HelpCircle,
 Link2,
 Loader2,
 Play,
 Plus,

```

```

 setStderr: (options: { batched: (message: string) => Promise<string> }) => Promise<string>
 setStdin: (options: { stdin: () => string | number | null }) => Promise<string>
};
```

```
type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideModule>
};
```

```
let runtimePromise: Promise<PyodideRuntime> | null = null;
```

```
function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL,
 { module as PyodideModule }).loadPyodide({ indexURL: indexURL });
 }
 return runtimePromise;
}
```

```
function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}
```

```
/* -----
/* Шаблон синхронизации: короткая шапка + минимальный скелет
/* скелет, шаг-в-шаг с подсветкой визуализации (см. data-visualizer.ts)
/* -----
```

```
/* -----
/* Шпаргалка Python: команды сортировки, циклы, структуры данных
/* «Вставить» кладёт сниппет в редактор под курсор.
/* -----
```

```
interface Snippet {
 label: string;
 tip: string;
 code: string;
}
```

```

],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = pop",
 code: 'st = []\nst.append("A") # push\nst.pop() # pop',
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n).",
 code: 'from collections import deque\nq = deque()\nq.append(1)\nq.popleft()',
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) – как очередь приоритетов",
 code: 'import heapq\nh = []\nheapq.heappush(h, (1, 'A'))\nheapq.heappop(h)',
 },
 {
 label: "Матрица n × m",
 tip: "Таблица расстояний: dist[i][j], бесконечность float('inf')",
 code: 'n, m = 3, 4\ndist = [[float("inf")] * m for _ in range(n)]',
 },
 {
 label: "Множество visited",
 tip: "Посещённые вершины: in/add за O(1).",
 code: 'visited = set()\nvisited.add("A")\nprint("A" in visited)',
 },
],
},
{
 group: "Вывод",
 items: [
 {

```

```

 savedEditor.code.trim() !== "" &&
 savedEditor.code !== template
 ? chapterId
 : null
);
 const prevTemplateRef = useRef(template);

 /** Активная отладочная сессия: трасса по шагам + текст
 const [debug, setDebug] = useState<{ result: DebugResult }>(null);
 const listRef = useRef<HTMLDivElement>(null);

 useEffect(() => {
 savedEditor = { code, template, chapterId };
 }, [code, template, chapterId]);

 // Смена страницы: нетронутый редактор получает свежий код
 // свой код не затираем – предлагаем синхронизировать
 useEffect(() => {
 if (template === prevTemplateRef.current) return;
 const prevTemplate = prevTemplateRef.current;
 prevTemplateRef.current = template;
 setDebug(null); // трасса ссылается на строки старого кода
 if (code === prevTemplate || code.trim() === "") {
 setCode(template);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [template]);

 const resync = useCallback(() => {
 prevTemplateRef.current = template;
 setCode(template);
 setDesyncedFrom(null);
 setStripTab("vars");
 setDebug(null);
 }, [template]);

```

```

 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать – бу
 }
}, [code]);

/* — Пошаговый отладчик —————

/** Редактор на шаблоне страницы: каждая строка трасси
const lockedToViz = code === template;

const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d) return d;
 const idx = Math.max(0, Math.min(d.result.steps
 if (code === template) emitVizStep(chapterId, v
 return { ...d, idx };
 });
 },
 [code, template, chapterId, stepLines]
);

const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {
 if (!d) return d;
 const idx = Math.max(0, Math.min(d.result.steps
 if (code === template) emitVizStep(chapterId, v
 return { ...d, idx };
 });
 },
 [code, template, chapterId, stepLines]
);

const stepTo = useCallback((idx: number) => goDebug(i

```



```

 }
 }, [code, stdin, running, runtimeReady, template, char

// Стрелки ← → в режиме отладки листают ШАГИ (а не стр
useEffect(() => {
 if (!debug) return;
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target))
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();
 e.preventDefault();
 stepBy(1);
 }
 if (e.key === "ArrowLeft") {
 e.stopPropagation();
 e.preventDefault();
 stepBy(-1);
 }
 if (e.key === "Escape") stopDebug();
 };
 // capture-фаза: чтобы глобальный обработчик навигац
 window.addEventListener("keydown", onKey, true);
 return () => window.removeEventListener("keydown", onKey);
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
const curDebug = debug ? { step: debug.result.steps[debug.result.step] } : undefined;
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
 if (curDebugLine === undefined || !listRef.current) return;
 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
}, [curDebugLine]);

// производные данные шага: что изменилось, в каком п
```

```

 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Скопировать код"
 >
 {copied ? <Check className="w-4 h-4 text-slate-400" /> : <Copy />}
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Вернуть комментарий" />
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Скрыть панель" />
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Заккрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip
 side="bottom"
 content="Пошаговый отладчик: выполняет код и листаются стрелками ← →."
>
 <button
 type="button"
 onClick={() => void debugRun()}
 disabled={running}
 className="inline-flex items-center gap-1"
 >

```

```

 >
 <HelpCircle className="w-3.5 h-3.5" /> Подс
 </button>
 <div className="ml-auto pr-1">
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Редактор и визуализация смотрят на
 : "На выбранной странице нет интеракт
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex item
 hasViz ? "text-emerald-400" : "text-s
 }` }
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет виз

 </Tooltip>
 </div>
</div>

{stripTab === "vars" ? {
 <div className="px-3 pb-2.5 space-y-2 max-h-3
 <div className="flex items-center gap-2 min
 <Tooltip side="bottom" content="Открыть э
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center g
700 text-slate-300 hover:text-white hover:b
 >
 {chapterTi
 </button>
 </Tooltip>

```

```

 >
 {v.name}
 </code>
 </Tooltip>
)))
 {sync.stepNote && {
 <Tooltip side="bottom" content={`War
 <span
 tabIndex={0}
 className="cursor-help inline-flex
text-[10px] text-slate-400 hover:text-white
 >
 <ArrowRightLeft className="w-3 h-3
 <span className="truncate max-w-5

 </Tooltip>
 }}
 </div>
)}
</div>
) : {
 <div className="px-3 pb-2.5 max-h-36 overflow
 {SNIPPETS.map(({gItem}) => {
 <div key={gItem.group}>
 <div className="text-[9px] font-bold up
 <div className="flex flex-wrap gap-1.5"
 {gItem.items.map(({s}) => {
 <Tooltip key={s.label} side="bottom
 <button
 type="button"
 onClick={() => insertSnippet(s.
 className="inline-flex items-ce
 -[10.5px] text-indigo-300 hover:text-white
 title=""
 >
 <Plus className="w-3 h-3 text-s
 {s.label}
 </button>

```

```

 <SkipForward className="w-4 h-4" />
 </button>
 </Tooltip>
 <span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? {
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>" &
 </>
 } : {
 "шагов нет – программа не выполнила н
 }}

 {debug.result.truncated && {
 <Tooltip content={`Трасса обрезана: лим
шие данные.`}>
 <span tabIndex={0} className="cursor-l
rounded-full px-1.5 py-0.5">
 лимит

 </Tooltip>
 }}
 <Tooltip
 content={
 lockedToViz
 ? "Код = шаблон страницы: каждая стр
 : "Код изменён пользователем: демон
к шаблону» / ресинк), чтобы восстановить св.
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex i
 lockedToViz
 ? "border-emerald-500/40 bg-emera
 : "border-slate-600 bg-slate-800
 }`}
 >

```



```
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between
 colors"
 >

 <Terminal className="w-3.5 h-3.5 text-emerald-500" />

 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" />}
 {outputOpen ? 'Закрыть' : 'Открыть'}

 </button>
 {outputOpen && (
 <pre className="flex-1 min-h-[70px] overflow-hidden leading-5 text-slate-300">
 {output}
 </pre>
)}
 <div className="px-3 py-1.5 border-t border-slate-200" />
 </div>
 </aside>
);
}
```

---

ФАЙЛ 59/87: `src/components/QueueViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
 id: string;
 name: string;
```

```

const addLog = (msg: string) => setLog(prev => [msg,

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяже.
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.
 }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск
 setTimeout(() => setShake(false), 400);
 return;
 }

```



```

 }, 500));
 };

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-in
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 tex
 <p className="text-xs text-slate-300 mt-1 lea
 В очереди за супом всё честно. Кто первый в
 Новые гости встают строго в конец оч
 Здесь нет обхода и блата — только строгий п
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-t
 d:opacity-40 disabled:cursor-not-allowed te
 ive:scale-95 transition-all cursor-pointer
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-t
 opacity-40 disabled:cursor-not-allowed text
 :scale-95 transition-all cursor-pointer fle
 >
 Налить суп первому (DEQUEUE)

```

```


 </div>
 </div>

 { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
 <div className="flex flex-col items-center" >

 </div>

 { /* СПИСОК ОЧЕРЕДИ */ }
 <div className="flex-1 flex items-end gap-4" >
 {queue.length === 0 ? {
 <div className="flex-1 flex flex-col" >
 </
 <p className="text-xs font-bold font" >
 <p className="text-[10px]" >Все сыты
 </div>
 } : {
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length - 1;
 return (
 <div
 key={customer.id}
 className={`
 flex flex-col items-center gap-2
 ${customer.animState === 'in' ? 'animate-in' : ''}
 ${customer.animState === 'eat' ? 'animate-eat' : ''}
 ${customer.animState === 'out' ? 'animate-out' : ''}
 `
 >
 { /* Облачко мыслей */ }
 <div className={`
 absolute -top-16 left-1/2 -transform translate-x-50%
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 border' : ''}
 `} >

```

```

 { /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
 <div className="md:col-span-3 bg-slate-900 rounded-3xl" >
 <div className="absolute top-4 left-4 text-[10px] font-mono" >
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-center gap-2" >
 <div className="order-emerald-800/80 text-[9px] font-mono" >
 <Sparkles className="w-3.5 h-3.5 animate-pulse" >
 СЫТЫ
 </div>
 </div>

 <div className="flex-1 flex flex-col justify-between" >
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-2" >

 <p className="text-xs font-bold font-mono" >
 <p className="text-[10px] mt-1" >Здесь 6
 </p>
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-full" >
 {servedHistory.map(({item, idx}) => (
 <div
 key={idx}
 className="w-full py-1.5 px-3 rounded-3xl" >
 0 flex items-center gap-2 text-[11px] text-
 >
 {item}
 </div>
)}}
 </div>
)}
 </div>

 { /* Пол */ }
 <div className="w-full h-3 bg-gradient-to-r from-emerald-500 to-teal-500" >
 </div>
 </div>

```

```

 y?: number;
 }

export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: T
 const [simulationStep, setSimulationStep] = useState<
 const [bfsQueue, setBfsQueue] = useState<number[]>([
 const [currentNode, setCurrentNode] = useState<number
 const [speechText, setSpeechText] = useState<string>('
 'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
 буль!')
);
 const [isTalking, setIsTalking] = useState<boolean>(f
 const [isBlinking, setIsBlinking] = useState<boolean>

 // Регулярное моргание рыбы
 useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
 }, []);

 // Эффект разговора при смене текста
 useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
 }, [speechText]);

 // Построение начального бора (Trie) без fail-ссылок
 const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {}
 };
 };

```

```

 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX;
 if (leftX === -1) currentX++;
 }
};

```

```

layoutDFS(0, 0);

```

```

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД) для слова "Базовое".

 Теперь можно расставить fail-ссылки!`
);

```

```

};

```

```

useEffect(() => {
 buildInitialTrie(words);
}, []);

```

```

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове "Базовое".`);
 return;
 }
 words.push(cleanWord);
 setWords(words);
 buildInitialTrie(words);
};

```

```

 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полный

 ь полюбоваться таблицей переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.cha

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] === undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTa
 }

 logs += `Для #${u} ('${char}') fail = #${failTarg
 }

 if (childrenIds.length === 0) {

```

```

 { /* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center justify-center"
 <div className="w-40 h-40 relative flex items-center justify-center"
 full border border-blue-500/30 shadow-inner
 { /* SVG Лосося */ }
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform duration-200ms
 isTalking ? 'scale-105 drop-shadow-[0_0_10px_#000]' : ''
 }`
 >
 { /* Тело лосося */ }
 <ellipse cx="100" cy="100" rx="70" ry="45" fill="#e67e22" />
 <path d="M 40 85 Q 100 55 160 85 Q 100 115 40 85" fill="#e67e22" />

 { /* Хвост */ }
 <path
 d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
 fill="#be123c"
 className="origin-[35px_100px] animate-pulse"
 />

 { /* Плавники */ }
 <path d="M 90 55 L 70 30 L 110 55 Z" fill="#000000" />
 <path d="M 90 145 L 70 170 L 110 145 Z" fill="#000000" />
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate-pulse"
 />

 { /* Глаз (с морганием) */ }
 <circle cx="145" cy="85" r="12" fill="#ff0000" />
 { isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85" stroke="black" stroke-width="2" />
) : (
 <circle cx="148" cy="85" r="6" fill="#000000" />
) }
 </svg>
 </div>
 </div>

```

```

 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-slate-200">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400" />
 Статус: {simulationStep === 0 ? 'Борется с вирусом!' : 'Восстановлен!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-4 py-2 shadow-lg shadow-emerald-900/40 transition-colors"
 >
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500 text-white px-4 py-2 shadow-lg shadow-blue-900/40 transition-colors"
 >
 Шаг BFS

 {simulationStep}

 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500 text-white px-4 py-2 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузить
 </button>
)}
 </div>
 </div>
 </div>
</div>

```



```

 const isCurrent = currentNode === node;

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x} / 2 - 10)}
 y={({parent.y + node.y} / 2)}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
 }
}

{/* Draw Fail Links */}
{simulationStep > 0 && Object.values(nodes).some(
 node => {
 if (node.id === 0 || node.fail === 0) return false;
 // n draw all! But maybe wait, we can just draw the fail link
 // Wait, all nodes get fail link. Let's draw it when simulationStep
 // step >= 1 and computed it
 // To avoid too many lines, let's draw the fail link only when it's
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !target.x || !target.y) return false;

 // Simple bezier curve for fail link

```

```

 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && (
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
)}
 </g>
);
 })}
</svg>
);
})();
</div>

{/* Управление словарем */}

```

```

 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
 </form>
 </div>

```

{/\* Таблица Вершин и суффиксных ссылок \*/}

```

<div className="lg:col-span-2 bg-slate-900/50 p-4">
 <h5 className="text-white font-bold mb-3 flex items-center">

 Таблица переходов и fail-ссылок

 Всего вершин: {Object.keys(nodes).length}

 </h5>

```

```

<div className="max-h-[220px] overflow-y-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-slate-700">
 <th className="py-2 px-3">ID Вершины</th>
 <th className="py-2 px-3">Символ</th>
 <th className="py-2 px-3">Родитель</th>
 <th className="py-2 px-3">fail-ссылка</th>
 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict)</th>
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800">
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node
 const isInQueue = bfsQueue.includes(node)

 return {

```

```

 #{node.fail}

 }}
 </td>
 <td className="py-2.5 px-3">
 {node.id == 0 || node.term_link
 <span className="text-slate-500"
) : {
 <span className="bg-rose-950 text-white"
0 transition-colors cursor-help" title={`Ск
 #{node.term_link}

 }}
 </td>
 <td className="py-2.5 px-3 text-slate-500">
 {Object.keys(node.children).length == 0
 ? 'Ø'
 : Object.entries(node.children)
 .map([c, id]) => `${c}→#${id}`}
 .join(', ')}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length == 0 ? (
 <span className="text-slate-600"
) : (
 <span className="bg-emerald-900 text-white"
 {node.words.join(', ')}

)}
 </td>
</tr>
</tbody>
</table>
</div>
</div>
</div>

```

```
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
```

```
 const n = arr.length;
```

```
 const steps: Step[] = [];
```

```
 let id = 0;
```

```
 const tree: Record<number, number> = {};
```

```
 const CODE = [
```

```
 "build(v, l, r) {", // 1
```

```
 " if (l === r) {", // 2
```

```
 " tree[v] = A[l]; return", // 3
```

```
 " }", // 4
```

```
 " mid = (l + r) >> 1", // 5
```

```
 " build(2v, l, mid)", // 6
```

```
 " build(2v+1, mid+1, r)", // 7
```

```
 " tree[v] = tree[2v]+tree[2v+1]", // 8
```

```
 "}", // 9
```

```
];
```

```
void CODE;
```

```
function go(v: number, l: number, r: number) {
```

```
 if (l === r) {
```

```
 steps.push({ id: id++, desc: `build(${v}, ${l}, $
```

```
 , highlightNodes: [v], arrayHighlight: [l, r] } }
```

```
 tree[v] = arr[l];
```

```
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[
```

```
 l, r] } });
```

```
 return;
```

```
 }
```

```
 const m = (l + r) >> 1;
```

```
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r
```

```
 { ...tree }, highlightNodes: [v], arrayHighlight:
```

```
 go(2 * v, l, m);
```

```
 steps.push({ id: id++, desc: `Вернулись в build(${v
```

```

 return res;
 }
 const ans = go(1, 0, n - 1, qL, qR);
 steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`,
 highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
 return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number): Step[] {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

 steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`,
 codeLine: 3, treeState: tree, highlightNodes: [l, r] });

 while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l`,
 codeLine: 4, treeState: tree, highlightNodes: [l] });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный → r`,
 codeLine: 5, treeState: tree, highlightNodes: [r + 1] });
 highlights.push(r + 1);
 }
 l = (l + 1) / 2;
 r = (r - 1) / 2;
 }
 return steps;
}

```

```

"query(v, l, r, ql, qr) {",
" if (ql > r || qr < l) return 0;",
" // -----",
" if (ql <= l && r <= qr) return tree[v];",
" // -----",
" mid = (l + r) >> 1;",
" // спускаемся в обоих детей",
" return query(left) + query(right);",
"}"
];

const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];

```

```

 [{nd.l
 <span className="text-xs font-extrabold font-
</div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full l
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
 </div>
);
}, [step, clr]);

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 }/* Header */>
 <div className={`px-4 py-2.5 flex items-center ju
 -emerald-950/50' : 'bg-amber-950/50'} border-b l
 <h4 className="font-bold text-sm text-white fle
 <span className="text-[10px] font-mono text-sla
 </div>

 {/* Array strip */}
 <div className="px-3 py-2 bg-slate-950 border-b b
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= s
 return (
 <div key={i} className={`flex flex-col item
 go' ? 'bg-indigo-950 border-indigo-500' : c
 er-500'} -translate-y-0.5` : 'bg-slate-900 l
 <span className="text-[7px] text-slate-50
 <span className="text-[11px] font-bold for
 </div>
);
 }
)}
 </div>
);
);

```



```

<div className="px-3 py-2.5 bg-slate-950 border-t
 <div className="flex items-center bg-slate-900
 <button onClick={() => { setPlaying(false); s
 unded" title="Сброс"><RotateCcw className="
 <button onClick={() => { setPlaying(false); s
 0 hover:text-white hover:bg-slate-800 round
 <button onClick={() => setPlaying(!playing)}
 white transition-all ${playing ? 'bg-rose-5
 {playing ? <Pause className="w-3 h-3" /> :
 </button>
 <button onClick={() => { setPlaying(false); s
 .5 text-slate-400 hover:text-white hover:bg
 utton>
 </div>
 <select value={speed} onChange={e => setSpeed(N
 text-[9px] font-bold rounded px-1.5 py-1 outl
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
</div>

 {/* Progress bar */}
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
</div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12
 const [customInput, setCustomInput] = useState('5, 8,
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);

```

```

 label>
 <button type="button" onClick={randomize}
"><RefreshCw className="w-3.5 h-3.5" /></bu
</div>
<div className="flex gap-2">
 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.v
 className="flex-1 bg-slate-900 border b
one focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo
ition-colors">Применить</button>
</div>
<div className="flex flex-wrap gap-1.5 pt-1
 {arr.map((v, i) => (
 <span key={i} className="bg-slate-900 b
 A[{i

)))}
 <span className="text-xs text-slate-500 f
</div>
</form>

{/* Query range */}
<div className="bg-slate-950 p-4 rounded-xl b
 <label className="text-[10px] font-bold upp
 <div className="flex items-center gap-2">
 <div className="flex items-center gap-1">
 <span className="text-xs text-slate-400
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(
 className="w-14 bg-slate-900 border b

```

```

 <Info className="w-3.5 h-3.5" /> Почему / Сравн
 </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && {
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
}}

```

```

{view === 'queries' && {
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`
 icon="↑ "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`
 icon="↓ "
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>

```

```

 span className="text-rose-400 font-mono">0(
 <div className="flex justify-between bo
>
 <div className="flex justify-between"><
d-400 font-bold">✓ легко</div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl
 <div className="absolute -top-3 left-4 bg
rder border-amber-500 shadow-md">
 r Запрос Снизу-вверх (Итерация)
 </div>
 <p className="text-xs text-slate-300 mb-3
 Стартуем с двух указателей (l и r) на л
ём его левого соседа. Поднимаемся пока l {"
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between bo
span className="text-emerald-400 font-mono":
 <div className="flex justify-between bo
><span className="text-emerald-400 font-mono
 <div className="flex justify-between"><
00 font-bold">x сложно</div>
 </div>
</div>
</div>

```

```

{ /* Verdict */ }
<div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="font-bold text-blue-400 mb-2
 <p className="text-sm text-slate-300 leadin
 <strong className="text-white">Итеративны
кальности. Но если нужны <strong className=
ame="text-white">рекурсивный (сверху-вниз)<
 </p>
</div>
</div>

```

```

 })

```

```

<aside className="w-full bg-slate-900/80 lg:bg-tran
 <div className="p-4 pb-3 shrink-0">
 <h3 className="text-xs font-bold text-slate-400">
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам..."
 className="w-full bg-slate-800/70 border border-slate-500 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-x-1/2 text-slate-500 hover:text-indigo-500"
 aria-label="Очистить поиск"
 />
 <X className="w-3.5 h-3.5" />
)}
 </div>
 </div>
 </div>

```

```

<div className="flex-1 overflow-y-auto px-3 pb-4">
 {groups.length === 0 && <p className="text-sm text-slate-500">Нет тем в избранном</p>}

 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold uppercase text-slate-500">{category}</div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 return (
 <button

```

Подчёркнутые термины в тексте раскрываются по  
симулятора.

```
</p>
 <p className="text-slate-400 italic">Стрелки ←
</div>
</aside>
);
};
```

---

ФАЙЛ 63/87: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar
```

```
interface Plate {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Person {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}
```

```
export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |
```

```

 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 });
 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate.name} `);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} `);
};

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🧑‍💻' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),

```

```

 };

 const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter(({_, idx}) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:
 };

 const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
 };

 // Sorted heap for display (Max-Heap property view)
 const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

 return (
 <div className="bg-slate-900/90 border border-slate-800 p-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-slate-400 to-slate-600">
 Интерактивный симулятор структур данных
 </h2>
 </div>
 </div>
);
 }
}

```



```

 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>
 <span className="text-xs bg-indigo-500/20 tex
 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 t
 : 'bg-slate-800/60 border-slate-700/60 te
 }`}
 >
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab ===
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">Приор
 </div>
 </div>
 <span className="text-xs bg-emerald-500/20 te
 Beam Search

 </button>
</div>

{/* STACK TAB */}
{activeTab === 'stack' && (
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция стопки тарелок

```



```

 <Plus className="w-4 h-4" /> Встать в очередь
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex items-center border-indigo-500/40 px-4 py-2.5 rounded"
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-indigo-500 transition-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
</div>

{dequeueMessage && (
 <div className="bg-indigo-950/60 border border-indigo-500/60 p-3 animate-fade-in">

 </div>
)}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2xl overflow-x-auto">
 {queue.length === 0 ? (
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3">0</div>
 <p className="text-base font-bold">Очередь пуста</p>
 <p className="text-xs mt-1">Добавьте сообщение в очередь</p>
 </div>
) : (

```

```

 </div>
);
 }}}

 <div className="flex flex-col items-center
-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
 </div>
}}
<div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
}}

{/* HEAP TAB */}
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 <div className="flex items-center gap-3 w-f
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex ite
py-2.5 rounded-xl text-sm font-bold shadow-

```

```

 onChange={e => setNewCustomProb(Number(
 className="w-full accent-emerald-500 cu
 />
</div>
<div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center jus
emerald-500/40 disabled:opacity-50 disabled:c
ointer h-10"
 >
 <Plus className="w-4 h-4" /> Добавить в
 </button>
</div>
</form>

{heapMessage && (
 <div className="bg-emerald-950/60 border bo
 gap-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {heapMessage}
 </div>
)}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
 >
 {sortedHeap.length === 0 ? (
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча
 <p className="text-xs mt-1">Добавьте ги
 </div>
) : (
 <div className="w-full max-w-2xl space-y-
 <div className="flex items-center justifi
 КАНДИДАТ / ГИПОТЕЗА

```

```
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%
 ></div>
 </div>
 <span className={`font-mono font
 isTop ? 'text-emerald-400 tex
 }`}>
 {percent}%

 </div>
 </div>
);
 }}}
</div>
)}
<div className="mt-6 text-xs text-slate-500
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
)}
</div>
);
};
```

---

ФАЙЛ 64/87: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";
```

```
interface Props {
```

здесь их можно открыть напрямую.

```
</p>
</div>

<div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absolute">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -transl
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
)}
 </div>

{items.length === 0 && <p className="text-slate-500">
 </p>
}

<div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-3">
 {items.map(({ id, entry, chapterTitle }) => (
 <div
 key={id}
 className="flex flex-col bg-slate-900 border border-
 >
 <h3 className="font-bold text-white text-sm">{chapterTitle}</h3>
 {entry.hint && <p className="text-[12px] text-white">{entry.hint}</p>
 && <div className="flex-1">
 <div className="flex items-center gap-2 mt-2">
 <button>
```

## Универсальное пособие • полный исходный код

-----  
\* поднять отмеченный узел в корень. Клик по узлу = один  
\* Порядок поворотов выбираешь сам; правильный ли он по  
\* только после того, как узел окажется наверху (или по  
\*/

```
export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null
```

```
const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
```

```
export const PRESETS: { name: string; hintCase: string;
```

```
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
```

```
 {
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
}
```



```

const x = path[path.length - 1];
const p = path[path.length - 2];
const gg = path.length >= 3 ? path[path.length - 3] : null;

if (p.left === x) {
 p.left = x.right;
 x.right = p;
} else {
 p.right = x.left;
 x.left = p;
}

if (!gg) return x;
if (gg.left === p) gg.left = x;
else gg.right = x;
return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number) {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя поворот)" };
}

/* ----- раскладка -----

interface Placed {

```

```

const [tree, setTree] = useState<TNode>(() => preset.t
const [history, setHistory] = useState<TNode[]>([]);
const [moves, setMoves] = useState<{ node: number; co
const [showAnswer, setShowAnswer] = useState(false);

const target = preset.target;
const solved = tree.key === target;

const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, tar
const expected = useMemo(() => (solved ? null : canon

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с ке

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);

```

```

)))
 </div>

 <div className="bg-slate-900 rounded-xl border bo
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420)
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-b
 />
);
 })}

 {nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n
 style={{
 transform: `translate(${n.x}px, ${n.y

```

```

 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-2
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-amber-500"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
 </button>

 поворотов: {moves.length} • минимум: {minimalMoves}

 </div>

 {showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40 p-3">
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-500"}
 >
 <div className="flex items-center gap-2 font-[12px]">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <X className="w-4 h-4" />}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} шагов`
 : `Узел ${target} в корне за ${moves.length} шагов`}
 </div>
 </div>
)}

```

```

* это и есть доказательство, что поворот ничего не
* * кадр «прицел» ничего не двигает, только подсвечива
* * на кадре «результат» остаются призраки – пунктирные
* местах и стрелки «откуда → куда»;
* * по умолчанию анимация НЕ крутится сама: пользователь
* идёт в своём темпе. Автопрокрутка включается кнопкой
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
 moved?: NodeId[];
 ms: number;
}
```

```
interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}
```

```

 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддеревов

 ева от 40 – это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба

 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый сл

 зглаживает.",
 ms: RESULT_MS,
 },
],
};

```

```

/* ----- Zig-Zag -----
const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p – дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым – нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по
 ",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,

```

```

 p: "p — родитель",
 g: "g — дед",
 };

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 /* призраки: где узел стоял до поворота */
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fil
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"

```



```

 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke=
<circle r={r} fill={c.fill} stroke={c.stroke}
<text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font=
 {id}
 </text>
)}
 </g>
);
 }}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

 const active = CASES[caseIdx];
 const total = active.frames.length;

```

```

<div className="mb-3 rounded-xl border border-slate-500"
 <div className="flex items-center gap-2 mb-1"
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`
 >
 {current.kind === "aim" ? "прицел" : current.title}

 <span className="text-sm font-bold text-white"
 >{idx + 1} из {total}. {current.title}
 </div>
 <p className="text-[13px] text-slate-400 leading-relaxed">
 </div>

```

```

<div className="bg-slate-900 rounded-xl border border-slate-500"
 <Tree frame={current} prev={prevPos} keys={activeKeys}>
 {/* обход слева направо – одинаковый на всех кадрах */}
 <div className="mt-2 pt-2 border-t border-slate-500"
 <div className="flex items-center gap-1.5 flex-wrap"
 Обход
 {active.inorder.map((t, i) => (
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-[10px]
 /\d/.test(t) ? "bg-emerald-500/15 text-emerald-300"
 : "bg-slate-700 text-slate-300"
 }`
 >
 {t}

))}


```

```

 {last ? "⏮ Сначала" : "Дальше →"}
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-3
 ext-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
 </button>
 <button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-3
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5x" : "1x"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto"
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (
 <span key={id} className="flex items-center gap-1"
 <span className="w-3 h-3 rounded-full" style={
 {background: active[id], border: "1px solid black"}
 } />


```

```

 } else if (key > root.key) {
 root.right = insertBST(root.right, key);
 }
 return root;
 };

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
 const y = x.left;
 if (!y) return x;
 x.left = y.right;
 y.right = x;
 return y;
};

// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
 const y = x.right;
 if (!y) return x;
 x.right = y.left;
 y.left = x;
 return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
 if (!node) return null;
 return {
 key: node.key,
 left: cloneTree(node.left),
 right: cloneTree(node.right),
 x: node.x,
 y: node.y,
 };
};

export const SplayTreeViz: React.FC = () => {
 // Let's create an elegant initial unbalanced or semi

```

```

 path.push({ node: current, dir: "left" });
 current = current.left;
 } else {
 path.push({ node: current, dir: "right" });
 current = current.right;
 }
}

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
 steps.push(`Поиск ${key}: элемент отсутствует в д...`);
 steps.push(
 `Поиск споткнулся на узле [${lastNode.key}]. И...`
);
} else {
 steps.push(
 `Поиск ${key}: элемент найден! Запускаем подъем...`
);
}

// Now let's implement standard Splay logic recursively
const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {
 if (!node || node.key === k) return node;

 if (k < node.key) {
 if (!node.left) return node;

 if (k < node.left.key) {
 // Zig-Zig (Left-Left)
 node.left.left = splayAction(node.left.left,
 node = rightRotate(node);
 steps.push(
 `Вращение Zig-Zig (Правый поворот родителя`
);
 }
 }
}

```

```

 return leftRotate(node);
 }
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
 setHighlightedNode(key);
 const { newRoot, steps } = splayStepByStep(tree, key);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree, val);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog([`Вставили вершину [${val}].`, ...steps]);
 setInputValue("");
 setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);

```

```

 <line
 key={`l-${node.key}-${node.left.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.left.x}
 y2={node.left.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.left));
 }
 if (node.right && node.right.x && node.right.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.right.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.right.x}
 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;
 };

const renderNodes = (node: SplayNode | null): React.R
 if (!node) return [];
 let circles: React.ReactNode[] = [];
 const isHighlighted = highlightedNode === node.key;

 circles.push(
 <g
 key={`n-${node.key}`}
 className="cursor-pointer group"

```

```

 </h3>
 <p className="text-xs text-slate-400">
 Кликните на вершины дерева, чтобы «вытащить»
 Splay.
 </p>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield */}
 <div className="lg:col-span-7 bg-[#0f172a] p-4" >
 <svg
 className="w-full h-full max-w-[600px] max-h-[320px]"
 viewBox="0 0 600 320"
 >
 {renderLines(drawTree)}
 {renderNodes(drawTree)}
 </svg>

 {/* Quick interactive controls */}
 <div className="absolute bottom-4 left-4 right-4" >
 <form onSubmit={handleFind} className="flex" >
 <input
 type="number"
 placeholder="Поиск..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target.value)}
 className="w-24 bg-slate-950 text-white"
 00 focus:outline-none"
 />
 <button
 type="submit"
 onClick={handleFind}
 className="bg-indigo-600 hover:bg-indigo-500"
 >
 Поиск / Splay
 </button>
 <button
 type="button"

```



```

 key={idx}
 className={`text-xs p-2.5 rounded trans
 idx === log.length - 1
 ? "border-l-2 border-indigo-500 bg-
 : "text-slate-400 bg-slate-900/50"
 }`}
 >
 {step}
 </div>
)))
 </div>

<div className="border-t border-slate-800 pt-1">
 <p>
 <span className="font-semibold text-sla
 у самого корня. Делаем 1 вращение.
 </p>
 <p>
 <span className="font-semibold text-sla
 оба левые или оба правые. Вращаем деда, п
 </p>
 <p>
 <span className="font-semibold text-sla
 колено. Вращаем узел в разные стороны.
 </p>
</div>
</div>
</div>

/* Answers Section inside the visualizer widget
<div className="bg-slate-950/80 p-5 border-t bord
 <h4 className="text-indigo-400 font-bold text-s
 <HelpCircle className="w-4 h-4 text-indigo-40
 Разбор экзаменационных вопросов (Без нитья):
 </h4>

 <div className="grid grid-cols-1 md:grid-cols-3
 <div className="bg-slate-900/60 p-4 rounded-l

```

```

 сложности <strong className="text-white"
 </p>
 </div>
 <div className="mt-3 text-[10px] text-rose-500">
 Кэшировать свингующие пары на концах – фа
 </div>
 </div>

 <div className="bg-slate-900/60 p-4 rounded-l
 <div>
 <p className="font-bold text-slate-300 mb-2">
 3. Амортизация $O(\log n)$
 </p>
 <p className="text-slate-400 leading-relaxed">
 Парадокс: один запрос за{" "}
 <code className="text-emerald-400"> $O(n)$ </code>
 всю структуру вдвое более плоской за счет
 при двойных вращениях {
 <code className="text-emerald-400 font-mono">
). Дорогая операция инвестирует в дешевые
 операций. Поэтому амортизированное время
 <strong className="text-white"> $O(\log n)$
 </code>
 </p>
 </div>
 <div className="mt-3 text-[10px] text-emerald-400">
 Каждое "растяжение" компенсируется "складыва
 </div>
 </div>
 </div>
</div>
);
};

```

```

const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>()

const addLog = (msg: string) => setLog(prev => [msg, ...prev])

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(`  PUSH: Положили ${newPlate.emoji} сверху стопки`);

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : { ...p, animState: 'out' }));
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 return;
 }

 const topPlate = dirtyStack[dirtyStack.length - 1];
 addLog(`  POP: Сняли ${topPlate.emoji} с тарелки`);
 setDirtyStack(prev => prev.slice(0, -1));
}, [dirtyStack]);

```

```

 mState: 'in' },
 });
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 });

 const topIndex = dirtyStack.length - 1;

 return (
 <div className="flex flex-col gap-6">
 { /* МНЕМОНИКА / ПРАВИЛО */ }
 <div className="bg-blue-950/40 border border-blue-950/40">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно самую верхнюю.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>
 </div>

 { /* УПРАВЛЕНИЕ */ }
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 text-white font-medium text-sm rounded-2xl disabled:cursor-not-allowed text-white
 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button

```

```

 <div className="absolute w-6 h-6 rounded-1
"></div>
 <div className="absolute w-4 h-4 rounded-1
"></div>
 <div className="absolute w-5 h-5 rounded-1
"></div>
 <div className="absolute text-4xl anim-cr
</div>
 }}

{dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {dirtyStack.slice().reverse().map(({plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div

```

```

 <div className="flex-1 flex flex-col justify-between"
 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </p>
 </div>
) : (
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full flex
items-center justify-center gap-2 opacity-100"
 >
 {plate.name}

 {plate.time}

 </div>
)
)}
 </div>
)}
 </div>

 {/* Столешница сушилki */}
 <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500"
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-700 p-4"
 <div className="text-[10px] font-mono text-slate-200"
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}
 >
 {idx === 0 ?

```

```

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре

```

```

const safePattern = pattern || " ";
const safeText = text || " ";

const { s, steps, patternLength } = useMemo(() => {
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
}, [mode, safePattern, safeText]);

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
useVizStepSync(stepIdx, setStepIdx, steps.length);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { setAutoPlay(false); }
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval(timer.current); };
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setStepIdx(stepIdx + 1); };
const prevStep = () => { setAutoPlay(false); setStepIdx(stepIdx - 1); };
const reset = () => { setAutoPlay(false); setStepIdx(0); };

const step = steps[stepIdx] || steps[0];

```



```

 let borderColor = "border-slate-900";
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "text-white" : "text-slate-900";

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include) {
 borderColor = "border-amber-500";
 bgColor = step.match === "KMP" ? "bg-amber-500" : "bg-slate-900/40";
 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-900";
 }
 if (step.zTarget === index) {
 borderColor = "border-amber-500";
 bgColor = step.match === "Z" ? "bg-amber-500" : "bg-slate-900/40";
 }
 }

 return {
 <div key={index} className={
 /* L/R markers for Z */
 {mode === "z" && step.l <= index && index <= step.r
 ? "border-2 border-amber-500"
 : ""}
 }
 {mode === "z" && step.r <= index && index <= step.l
 ? "border-2 border-amber-500"
 : ""}
 {/* Index */}
 <span className={`text-${

```

```

 })
 {mode === "z" && step.z
 <div className="abs
 <span className
 </div>
 })
 {mode === "z" && step.z
 <div className="abs
 <span className
 </div>
 })

 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 </div>
)
 }}}
</div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled
rounded-lg disabled:opacity-30 disabled:hov
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={playPause} classNam
x items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16} <
 </button>

```

```
 <div className="bg-slate-900 border border-
 <div className="bg-slate-800 px-4 py-2 b
late-400 uppercase">
 Код C++ ({mode === 'kmp' ? 'КМП'
 </div>
 <pre className="p-4 text-xs font-mono te
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}`}
 </pre>
 </div>
 </div>
);
 }
 }
}
```

---

ФАЙЛ 70/87: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from "react";
```

```
interface TooltipProps {
 /** Содержимое подсказки (можно JSX). */
 content: React.ReactNode;
}
```

```
 />

);
};
```

---

ФАЙЛ 71/87: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync } from '../data/vizStepBus';
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";
```

```
interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}
```

```
interface TEdge {
 u: number;
 v: number;
}
```

```
const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
];
```

```

 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
};

recordStep(
 "Начало топологической сортировки. Используем кла
 null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} (${dagNodes[u].name
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} о
 u,
);
 }
 }
}

fullyProcessed.add(u);
dfsStack.pop();

```

```

 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? {

```

```

 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
</defs>

{/* Edges */}
{dagEdges.map(({edge, idx}) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v);

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#3341
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar
 className="transition-all duration-300
 />
);
}})
}}}

{/* Nodes */}
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === no
 const isVisited = step.visited.has(node.id
 const isProcessed = step.fullyProcessed.h

```

```

 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" ")
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
 }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 text-slate-100 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800 p-4">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>


```



```

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
t-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
t-white"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
 </div>
</div>
);
};

```

ФАЙЛ 72/87: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";

import { ArticulationPointsViz } from "../ArticulationPo
import BellmanFordViz from "../BellmanFordViz";
import BfsViz from "../BfsViz";
import { DPVisualizer } from "../DPVisualizer";
import { DfsBridgesSimulator } from "../DfsBridgesSimula

```

```

 Разобрался – проверь себя: ниже то же самое дер
 не решишь.

</p>
<SplayRotationSandbox />
</div>
);

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки – это ровно то, ч
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {

```

```

 },
 "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphDfsBfs },
 "top-sort": { title: "Топологическая сортировка", Component: TopSort },
 "scc-kosaraju": { title: "Компоненты сильной связности", Component: SccKosaraju },
 "graph-articulation": { title: "Точки сочленения", Component: GraphArticulation },
 "bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesCode },
 "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPathVsCycle },
 "planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: PlanarityEulerFormula },
 dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
),
 },
 },
 "bellman-ford": {
 title: "Форд-Беллман",
 Component: () => (
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
),
 },
 floyd: {
 title: "Флойд-Уоршелл",
 Component: () => (
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
),
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonAlgo },
 "mst-kruskal": { title: "Краскал и DSU", Component: Kruskal }
}

```

```

 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: false, words: [] },
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
 8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка S -> H -> E
 10: { id: 10, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3, fail: 2 (HE), term_link: 0, is_terminal: false, words: [] },
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
 10: {}
};

```

```

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен!');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У русла #${curr} (${WATERFALL_NODES[curr].label})`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Лосось сбился с толку!`;
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} сбился с толку!`;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
 }
 }
}

```

```

 desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'Ø',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на De
 ,
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
 а-разведчик плывет в ROOT и ищет ветку по "E".
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,

```

```

 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) – КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTra

```

```

 Режим 2: Поиск в тексте
 </button>
 </div>
 </div>

 { /* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */ }
 { activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3" >
 { /* Управление шагами обучения */ }
 <div className="bg-slate-900/90 p-5 rounded-x" >
 <div>
 <h5 className="text-indigo-300 font-bold" >
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3" >
 Показаны все шаги очереди BFS (начиная с
 ар 8 (SHE → HE), где наглядно видно
 </p>
 <div className="space-y-1.5 mb-6 overflow" >
 { TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p-1
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font-medium'
 : 'bg-slate-800 text-slate-400'
 }` }
 >
 {step}
 {idx === trainStep &&

 }
 </button>
)}
 </div>
 </div>
 </div>
 }
 }

```



```

 </div>
 <div className={`p-1.5 rounded flex item
-l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail}.label}}
 {currentTrainInfo.codeLine === 2 && <
к родителю}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
/s>
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold ml
 <Lightbulb className="w-4 h-4 text-ambe
 </h5>
 <div className="bg-slate-800 p-4 rounded-:
w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>

```

```

 >
 {char}

)))
 {currentIndex >= textToScan.length && (

 ГOTOBO ✓

)}
 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium transition-colors
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 `}
 >
 {currentIndex === 0 ? 'Начать план' : 'Продолжить'}
 <ArrowRight className="w-4 h-4" />
 </button>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4" >
 <div>
 <h5 className="text-white font-bold mb-3">
 ✂ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-lg">
 <div className={`p-1.5 rounded flex items-center border border-blue-500 text-blue-300 font-bold`} >
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && инициализация}
 </div>
 </div>
 </div>
</div>

```

```

 { /* Водопадные фоновые эффекты */ }
 <div className="absolute inset-0 opacity-10 bg-
 o-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -transl
 <div className="w-8 bg-blue-400 h-full animat
 <div className="w-12 bg-blue-300 h-full anima
 <div className="w-6 bg-blue-500 h-full animat
 </div>

 <div className="flex items-center justify-betwe
 <h5 className="text-white font-bold text-base
 Ветвящийся синий водопад (Бо
 </h5>
 <div className="flex flex-wrap gap-4 text-xs"
 <span className="flex items-center gap-1 te
 <span className="w-3 h-0.5 bg-blue-500 in

 <span className="flex items-center gap-1 te
 <span className="w-3 h-0.5 border-t borde

 </div>
 </div>

 { /* Само SVG дерево водопада */ }
 <div className="w-full overflow-x-auto custom-s
 <svg viewBox="0 0 800 520" className="w-full

 { /* Горизонтальные линии и подписи уровней
 {[
 { depth: 0, y: 60, label: 'Depth 0 (Корень
 { depth: 1, y: 160, label: 'Depth 1 (H, S
 { depth: 2, y: 260, label: 'Depth 2 (HE, I
 { depth: 3, y: 360, label: 'Depth 3 (HER,
 { depth: 4, y: 460, label: 'Depth 4 (HERS
]}.map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"

```



```

 /* Рендер вершин (бассейнов водопада) */
 Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target';
 const hasWords = node.words.length > 0;

 return (
 <g key={`node-${node.id}`} className={isCurrent ? 'active' : ''}>
 /* Внешнее свечение для активной вершины */
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={node.radius} fill="none" stroke="black" stroke-width={2}/>
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={node.radius} fill="none" stroke="red" stroke-width={2}/>
)}

 /* Основной круг вершины */
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24px' : '18px'}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f87171' : 'white'}
 stroke={isCurrent ? 'white' : isTargetChild ? 'red' : 'black'}
 stroke-width={isCurrent || isTargetChild ? 2 : 1}
 className="transition-all duration-300"
 />

 /* Текст внутри вершины */
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : 'black'}
 fontSize={isCurrent || isTargetChild ? 14 : 12}
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>
 </g>
);
 });

```

```

 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezier(0.4, 0.6, 0.2, 0.8)' }}
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' : ' ♀ '}
 </text>
 </g>

 </svg>
 </div>
</div>

{/* Список найденных совпадений (только в режиме поиска) */}
{activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded-lg">
 <h5 className="text-white font-bold mb-3 text-sm">
 ⚠ Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic">
 Пока ничего не поймали. Запустите шаги симуляции.
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-800 text-white text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
 >
 <CheckCircle2 className="w-4 h-4 text-emerald-400"/>
 {match.word}

 Индекс {match.index}

 </div>
))}
 </div>
)}
 </div>
)}

```

```

 [9, 1, 2, 3],
 [4, 5, 6, 7],
];

const cellBase =
 "flex items-center justify-center rounded-md font-mono" +
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border" >
 <div className="flex gap-2 mb-4 overflow-x-auto" >
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm:text-sm
 ${mode === key ? "bg-indigo-600 text-white" : ""}
 `}
 >
 {label}
 </button>
))}
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(

```

```

 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r
 className={`${cellBase} cursor-pointe
 inCover
 ? "bg-indigo-500 text-white ring-1
 : inRange
 ? "bg-indigo-900/70 text-indigo
 : "bg-slate-800 text-slate-300
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
 })}
</div>

```

```

 {/* массив P */}
 <div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" &&
 const isL = i === range.l;
 const isR = i === range.r + 1;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 className={`${cellBase} cursor-help $
 active
 ? "bg-emerald-500 text-white ring
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"

```



```

 подвинуть границу запроса.
 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b className="text-indigo-400">у
 у <b className="text-indigo-400">a – ку
 </p>
)}
 </div>

 { /* Запрос */ }
 <div className="bg-slate-900 border border-indigo-400">
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base text-slate-500">
 $\text{sum}(a[\text{range.l}..\text{range.r}]) = P[\text{range.r} + 1]$
 {pref[range.r + 1]}
 {pref[range.l]}
 {pref[range.r + 1] - pref[range.l]}
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за $O(1)$ по префикс-сумме
 </p>
 </div>
</div>
);
};

```

```

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array.from(
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {

```

```

 setRect((r) =>
 i < r.r1 || j < r.c1 ? { r1:
)
 }
 className={`${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`
 >
 {v}
 </div>
);
 }}}
</div>
)}}
</div>
</div>

```

```

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {P.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hov
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
 return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i,

```

```
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычтенный
 </p>
 </div>
</div>
);
};
```

---

ФАЙЛ 75/87: src/components/visualizers/SparseTableViz.t  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 709 | РАЗМ

---

```
import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j - 1)][j - 1]);
 }
}
```

```

 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
}
}
}
}
}

```

```

export const SparseTableViz: React.FC = () => {
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">("1d");

 return (
 <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
 <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
 <button
 onClick={() => setTab("1d")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : "bg-slate-800 text-slate-400"}
 >
 1. Сворачивание 1D
 </button>
 <button
 onClick={() => setTab("2d_build")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : "bg-slate-800 text-slate-400"}
 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white` : "bg-slate-800 text-slate-400"}
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

```

```

 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
 </div>
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
 <div className="text-[11px] uppercase tracking-wide">
 <div className="flex gap-1 sm:gap-1.5 overflow-x-auto">
 {arr1D.map((v, idx) => {
 const inCover = !!covered && idx >= covered - 1;
 return (
 <div key={idx} className="flex flex-col items-center justify-center">
 <div
 className={`flex items-center justify-center w-[44px] h-[clamp(26px,8vw,44px)] text-[clamp(10px,1.2em,14px)]`}
 style={{ background: inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-slate-300" }}
 >
 {v}
 </div>
 </div>
);
 })}
 </div>
 </div>
 </div>

```

```

</div>
<div className="text-center font-bold text-
rap">
 2^0 (L=1)
</div>
<div className="text-center font-bold text-
rap">
 2^1 (L=2)
</div>
<div className="text-center font-bold text-
rap">
 2^2 (L=4)
</div>
<div className="text-center font-bold text-
rap">
 2^3 (L=8)
</div>

{/* Matrix */}
{Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mon
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const isVisible =
 isValid &&
 (j === 0 ||
 computeSteps.findIndex((s) => s.i
 step));

 // Active computing state
 let isActive = false;
 let isSrc1 = false;
 let isSrc2 = false;

 if (step > 0 && step <= maxStep) {

```

```

 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"
 }` }
 >
 {st1D[i][j]}
 </motion.div>
)}

```

```

{/* Arrows visualization */}
{(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <<
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc1 ? "0 0" : "40 0"}`}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#f59e00"}
 strokeWidth="3"
 />
 </motion.svg>
)}
</div>
);

```

```

) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)}
 </div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: num
 const [locked, setLocked] = useState<{ r: number; c: n

 const L = 1 << k;
 const activeCell = locked || hover;

 return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 s
 <h3 className="text-xl font-bold text-indigo-400">
 <p className="text-sm text-slate-400 mb-6 text-
 Для наглядности показываем только <b classNam

 <span className="text-rose-300 animate-pulse m
 иц!
 </p>

 <div className="flex bg-slate-950 p-1 rounded-l
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setHover(nul
 className={`px-4 py-2 rounded-md text-sm
 t-slate-400 hover:text-white hover:bg-slate
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s

```







```

 </div>
 })
 </div>
 </div>
</div>
);
};

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

 const H = r2 - r1 + 1;
 const W = c2 - c1 + 1;
 const kx = Math.floor(Math.log2(H));
 const ky = Math.floor(Math.log2(W));
 const Lx = 1 << kx;
 const Ly = 1 << ky;

 const matRows = 8 - Lx + 1;
 const matCols = 8 - Ly + 1;

 return (
 <div className="flex flex-col xl:flex-row gap-6"
 <div className="flex-1 min-w-0 bg-slate-900"
 <h3 className="text-xl font-bold text-ro
 <p className="text-slate-400 text-[13px]
 еpx, r2, c2 - правый низ).</p>

 <div className="flex flex-col items-cent
 <div className="grid grid-cols-[repea
 ative shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_,
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r

```

```

 <div className="bg-slate-950 p-4 rounded-
er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-center
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 <label className="flex items-center
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 </div>
</div>
</div>

<div className="flex-1 bg-slate-900 p-6 rounded-
 <h3 className="text-xl font-bold text-em
 <p className="text-sm font-sans text-sla
 Мы знаем, что максимальная степень дв
rounded text-white">{Lx}. Аналогично
te">{Ly}.
 </p>
 <p className="text-sm font-sans text-sla
 Чтобы покрыть весь прямоугольник, мы
Name="font-mono bg-[#0a0f1e] px-1 rounded b
роса. (Цветные прямоугольники слева).
 </p>

 <div className="bg-[#0a0f1e] rounded-xl

```

```

<h4 className="text-slate-400 font-bo
 Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{kx}][{ky}]
</h4>
<div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky].toString();

 let badge = null;
 if (isTL) { color = "bg-rose-500 text-white";
0 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-lime-500 text-white";
ue-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-500 text-white";
-emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-500 text-white";
mber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color}
tion-all duration-300 relative shrink-0 ${c === c1 ? 'w-1/2' : 'w-1/4'} h-1/4
 {txt}
 {badge && <div className="bg-slate-700 text-white px-1 font-bold rounded
 </div>
)
 })
 </div>
</div>

```

```

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').className
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 document.getElementById('btn-mode-wtf').className
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 const aside = document.querySelector('aside');
 const headerMain = document.querySelector('header');
 const questionsContainer = document.getElementById('questions-container');
 const wtfContainer = document.getElementById('wtf-container');

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode');
 if (aside) aside.style.display = 'none';
 if (headerMain) headerMain.style.display = 'none';
 if (questionsContainer) questionsContainer.style.display = 'none';
 if (wtfContainer) wtfContainer.classList.remove('hidden');

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname + window.location.search);
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden');
 }
 }

 function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light');
 }

```

```

 drawerOverlay.classList.remove(
 // Timeout allows display:block
 setTimeout(() => drawerOverlay.
 }
 document.body.style.overflow = "hid
} else {
 mobileDrawer.classList.add("-transl
 if (drawerOverlay) {
 drawerOverlay.classList.add("op
 setTimeout(() => drawerOverlay.
 }
 document.body.style.overflow = "";
}
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click",
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click"
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click",
}

// Close on link click
document.querySelectorAll("#mobile-drawer a
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.c
 toggleDrawer();
 }
 }));
});
});

```

```

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
}

```

```

<script src="https://cdn.tailwindcss.com"></script>
<script src="https://polyfill.io/v3/polyfill.min.js"></script>
<script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.2/es5/tex-mml-chtml.js"></script>
<script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\]']],
 },
 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?family=Fira+Code&display=block');
 @reference './src/index.css';
 .math-font { font-family: 'Fira Code', monospace; }
 .uno-card {
 width: 70px; height: 105px; border-radius: 10px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
 align-items: center; justify-content: center; text-align: center;
 color: white; overflow: hidden; flex-shrink: 0;
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #000, 1px -1px 0 #000;
 font-family: 'Arial Black', Impact, sans-serif;
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute; top: 50%; left: 50%;
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #000, 1px 1px 0 #000;
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
 .uno-card.mini { width: 45px !important; height: 60px !important; }
 .uno-card.mini::before, .uno-card.mini::after {
 font-size: 0.8em;
 }

 /* Global formula scroll settings */
 .formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 }

```



```

 px; }
 }
 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }
 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card-
 .card-green { background-color: #339e35; } .card-
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 for

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-ligh
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-ligh
 important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-

```

```

 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 ТЕМ
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">
 <button onclick="setMode('normal')" id="btn-normal">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf">WTF
 </div>
 </div>
 <div class="flex items-center gap-3">
 ТЕМА
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 transition-colors">
 <button onclick="setTheme('dark')" id="btn-dark">Темная</button>
 <button onclick="setTheme('light')" id="btn-light">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook">Тетрадь
 </div>
 </div>
 </div>

</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">

```

```

 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="20"
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#333"
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55"
 <circle cx="25" cy="50" r="15" fill="#f00"
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="10"
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform
 <svg xmlns="http://www.w3.org/2000/svg" class="
 <path stroke-linecap="round" stroke-linejoin
 </svg>
</button>

<!-- Затемнение фона на мобилках при открытом меню
```

```
<details class="group">
 <summary class="list-none cursor-pointer-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-base">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-teal-500 pl-3 font-bold text-teal-300">
 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text-base">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-amber-500 pl-3 font-bold text-amber-300">
 4. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-base">

```



```


нственности НОД

ПИРАЛЬ)

вращается!)
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно

 </summary>
 <div class="pl-6 space-y-2 text-blue-500">

деление)
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-500">
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text-blue-500">

 </div>
</details>
```

```


 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-tea

 <a href="#question-18" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-blue

 <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
 <span class="font-bold text-eme

 <a href="#question-20" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-ros

 <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
 <span class="font-bold text-ind

 <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuc

 <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m

```

```

 // Highlight active
 document.querySelector(
 a.style.fontWeight
 if (a.href.endsWith(
 a.style.fontSize
)
 });

 // Open the parent
 const corresponding
 if (correspondingLink
 const details =
 if (details &&
 details.open
 document.querySelector(
 if (other
 other
)
 });
 }
 }
});

}, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
});

sections.forEach(section => {
 observer.observe(section);
});
});

function fallbackModal(contentRaw,
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';

```



```
const btnContainer = document.c
btnContainer.style.display = 'f
btnContainer.style.gap = '10px'
btnContainer.style.marginTop =
```

```
const copyBtn = document.create
copyBtn.innerText = 'Копировать'
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor =
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5
copyBtn.style.cursor = 'pointer'
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.wri
 copyBtn.innerText = 'Ск
 } else {
 textarea.select();
 document.execCommand('c
 copyBtn.innerText = 'Ск
 }
 setTimeout(() => copyBtn.inn
};
```

```
const closeBtn = document.creat
closeBtn.innerText = 'Заккрыть';
closeBtn.style.padding = '10px'
closeBtn.style.backgroundColor :
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '!
closeBtn.style.cursor = 'pointe
closeBtn.onclick = () => modal.
```

```
box.appendChild(header);
box.appendChild(subHeader);
```

```

 const link = document.createElement('a');
 link.href = url;
 link.download = 'vyisshaya_algebra.pdf';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-900');

 document.getElementById('theme-dark').checked = theme === 'dark';
 document.getElementById('theme-light').checked = theme === 'light';
 document.getElementById('theme-auto').checked = theme === 'auto';

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-900');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `
 body.bg-white section h1 { color: #000000; }
 body.bg-white main { color: #000000; }
 body.bg-white main p { color: #000000; }
 /* Universal text color */
 body.bg-white .text-slate-900 { color: #000000; }
 body.bg-white .text-slate-900 { color: #000000; }
 body.bg-white .text-slate-900 { color: #000000; }
 `;
 }
 }

```

```

 body.bg-white .text-cyan
 body.bg-white .text-amber
 body.bg-white .text-fuchsia
 body.bg-white .text-lime
 body.bg-white .text-blue
 body.bg-white .text-green
 body.bg-white .text-indigo
 `;
 } else if (theme === 'terminal') {
 body.classList.add('bg-black')
 document.getElementById('theme')
 .style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: transparent;
 }
 body.bg-black section h1 {
 color: #4ade80 !important;
 border-color: #22c55e !important;
 font-family: monospace;
 }
 body.bg-black svg path,
 body.bg-black button {
 stroke: #22c55e !important;
 fill: transparent !important;
 }
 `;
 }

 document.head.appendChild(style)
}
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo

```

```

 </div>
 </aside>

 <!-- Основной контент -->
 <main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
 -base min-w-0">
 <header class="text-center mb-12 border-b border-white" >
 <h1 class="text-4xl md:text-5xl font-bl
 4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg" >
 Высшая алгебра

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-center" >
 <h2 class="font-bold text-xl mb-4">Print Table of Contents
 <ul class="text-base space-y-2">
 1. Алгебра
 2. Коммутативная алгебра
 3. Топология
 4. Статистика
 5. Группы
 6. Декартовы координаты
 7. Алгебраические структуры
 8. Клеймо
 9. Коммутативная алгебра
 10. Алгебраические структуры
 11. Алгебраические структуры
 12. Алгебраические структуры
 13. Алгебраические структуры
 14. Алгебраические структуры
 15. Алгебраические структуры
 17. Алгебраические структуры
 18. Алгебраические структуры
 19. Алгебраические структуры
 20. Алгебраические структуры
 21. Алгебраические структуры

```

```

-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-10" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 3: НОД и НОК / Алгоритм Ев
 <a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-12" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структур
 <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange

```

```

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-500
 <div class="font-bold text-slate-700

 <a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-500
 <div class="font-bold text-slate-700

 <a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emerald-500
 <div class="font-bold text-slate-700

 <a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
 <div class="text-xs text-rose-500
 <div class="font-bold text-slate-700

 <a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo-500
 <div class="font-bold text-slate-700

 <a href="#question-22" onclick="setl
-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia-500
 <div class="font-bold text-slate-700

 <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ

```

---

```

 <span class="text-w
k$).
 <span class="text-w
 <span class="text-w
ово.

</div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-fuch
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 sp
 <span class="text-w
i>
 <span class="text-w
т.д.).
 <span class="text-w
что так и задумано: "Ой, противоречие, теор

</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-eme
 <p class="text-sm text-slat
 <ul class="list-disc pl-5 sp
 <span class="text-w
, r_2$).
 <span class="text-w
 <span class="text-w
счет ограничений (размер остатка) докажи, ч

</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-yel

```

```
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
$с$. Пока получается 0 – корень жив. Как то
корня минус один.</p>
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
ициентов (они делятся на p), кроме старше
зложить.</p>
 </div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
. Чаще всего после этого Фейс-контроль p .
 </div>
</div>
```

```
 <h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
 <p class="text-sm text-slate-400">
тношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
 <ul class="space-y-4 pb-10">
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400">
 <p class="text-sm text-slate-400">
 <code class="text-x">
 </div>
```



---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 78/87: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 79/87: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили зашиты внутрь, интер
 * его можно открыть с флешки, отправить в мессенджер и
 * Живые React-визуализации в статику не переносятся —
 * подставляется заметка со ссылкой на онлайн-версию, а
 * которые в приложении всплывают по наведению, собираю
 * в конце документа.
```

```

.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; f
.export-foot { margin-top: 3rem; border-top: 1px solid
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important
 pre { white-space: pre-wrap; }
}
`;
```

```
const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных тер
 */
```

```
function prepareBody(html: string): { body: string; term
 const host = document.createElement("div");
 host.innerHTML = html;

 try {
 annotateTerms(host);
 } catch {
 /* если браузер не осилил регулярку – выгружаем тек
 }
```

```
const ids: string[] = [];
host.querySelectorAll<HTMLElement>(".term-hint").forE
 const id = el.dataset.termId;
 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
```

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, init
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercas
 <h1 style="color:#fff;font-size:1.6rem;font-weight:
 <div style="font-size:.72rem;color:#475569;margin-t
 </header>

 <div class="export-note">
 Это автономная копия: стили внутри файла, интернет
 Интерактивные тренажёры и анимации в статичный HTML
 они живут в онлайн-версии: <a href="${escapeHtml(op
 </div>

 <article class="prose prose-invert max-w-none">
${opts.body}
 </article>

${opts.gloss}

 <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}
```

```
function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
```

```

 chapters.forEach((c) => {
 const { body, termIds } = prepareBody(c.content);
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="
<p style="text-align:right;font-size:.72rem;margin:0 0 10px 0;">
 `);

 const body = `<nav id="toc" style="background:#0f172a;
 color:white;padding:10px 0 0 10px;">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:800;margin:0 0 10px 0;">
 <ol style="margin:0;padding-left:1.1rem;font-size:.85rem;">
</nav>
 ${bodies.join("\n")}`;

 const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms),
 origin: window.location.origin,
 });
 triggerDownload(html, "posobie-vse-temy.html");
 }
 }

```

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

ФАЙЛ 80/87: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМЕР: 1.5 КБ

```

 title: c.title,
 num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 });
});

const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
 console.log("| --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(`  • ${r.title} [${r.num || "-"}]`));
 console.log();
 };
};

```

```

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi
 .replace(/<[^>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) con
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlight
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0,
);
}

```

```
const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный к
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width:
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(` выход: ${path.relative(ROOT, PDF_PATH)
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

```
 [/^scripts\\/, "Скрипты сборки и экспорта"],
 [/^\.github\\/, "CI / автоматизация"],
 [/^[^/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel));

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) {
 continue;
 }
 if (e.isDirectory()) {
 walk(abs);
 } else {
 files.push(abs);
 }
 }
 };
 walk(ROOT);
 }
}
```



```

 });
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { time: true });
 const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ ИСХОДНОГО КОДА)");
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:          ${files.length}`);
 push(`Суммарный объём кода:              ${humanSize(totalBytes)}`);
 push(`Глав/билетов в пособии:            ${chapters.length}`);
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
 push(HR);
 chapters.forEach((c, i) => {
 push(`    ${String(i + 1).padStart(3, " ")}. ${c.title}`);
 });
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
 push(HR);
 let current = null;
 files.forEach((rel, i) => {

```

```

fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`      файлов:  ${files.length}`);
console.log(`      глав:    ${chapters.length}`);
console.log(`      строк:   ${txt.split("\n").length}`);
console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

ФАЙЛ 84/87: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

```

/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают в бакет) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

```

```
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
);
}
```

---

ФАЙЛ 85/87: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 3.5 КБ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT,
 PAGES_DIR,
 TXT_PATH,
 PAGE,
 ensureDir,
 humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(/<\/pre>
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/<\/pre>
```

```

 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 10));
let done = 0;
let cursor = 0;

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`отрендерено ${done} страниц\n`);
 }
 }
}

await Promise.all(Array.from({ length: concurrency }, worker));

```

```
push:
 branches:
 - main
 # Позволяет проверить Pages прямо из рабочей ветки
 - "arena/**"
workflow_dispatch:

permissions:
 contents: read
 pages: write
 id-token: write

concurrency:
 group: pages
 cancel-in-progress: true

jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build
```

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда

# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умол

required: false

default: "150"

concurrency:

group: rebuild-pdf-\${{ github.ref }}

cancel-in-progress: true

permissions:

contents: write

jobs:

- name: Step 2 – txt → png  
env:  
  EXPORT\_DENSITY: \${github.event.inputs.density}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
  {  
    echo "### Экспорт пересобран"  
    echo ""  
    echo "| Артефакт | Размер |"  
    echo "| --- | --- |"  
    echo "| \`public/export/full\_code\_all\_pages`"  
    echo "| \`public/export/full\_code\_all\_pages`"  
    echo "| PNG-страниц | \${ls tmp/export-pages}"  
  } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
  name: full-code-export  
  path: |  
    public/export/full\_code\_all\_pages.pdf  
    public/export/full\_code\_all\_pages.txt  
  if-no-files-found: error  
  retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v7  
with:  
  name: full-code-pages-png